Diffusion models for simulating distributions of observables

Korl Železnikar,

January, 2026

Introduction

- Tested models: stohastic DDPM + variants and (all?) SDE/score models: NCSN, VP, subVP, VE, DDIM, EDM, EDM2.
- Most models underperform, two show „promising“ results.

- Model details in the supplementary info section.

DDPM/stohastic frameworks

- Reverse sampling stochastically from Gaussians; the models are preconditioned to learn added noise.
- Learning variance unstable, with no visible improvements over learning only the mean.
- Cosine schedule for the forward process was used.

Algorithm 1 Training

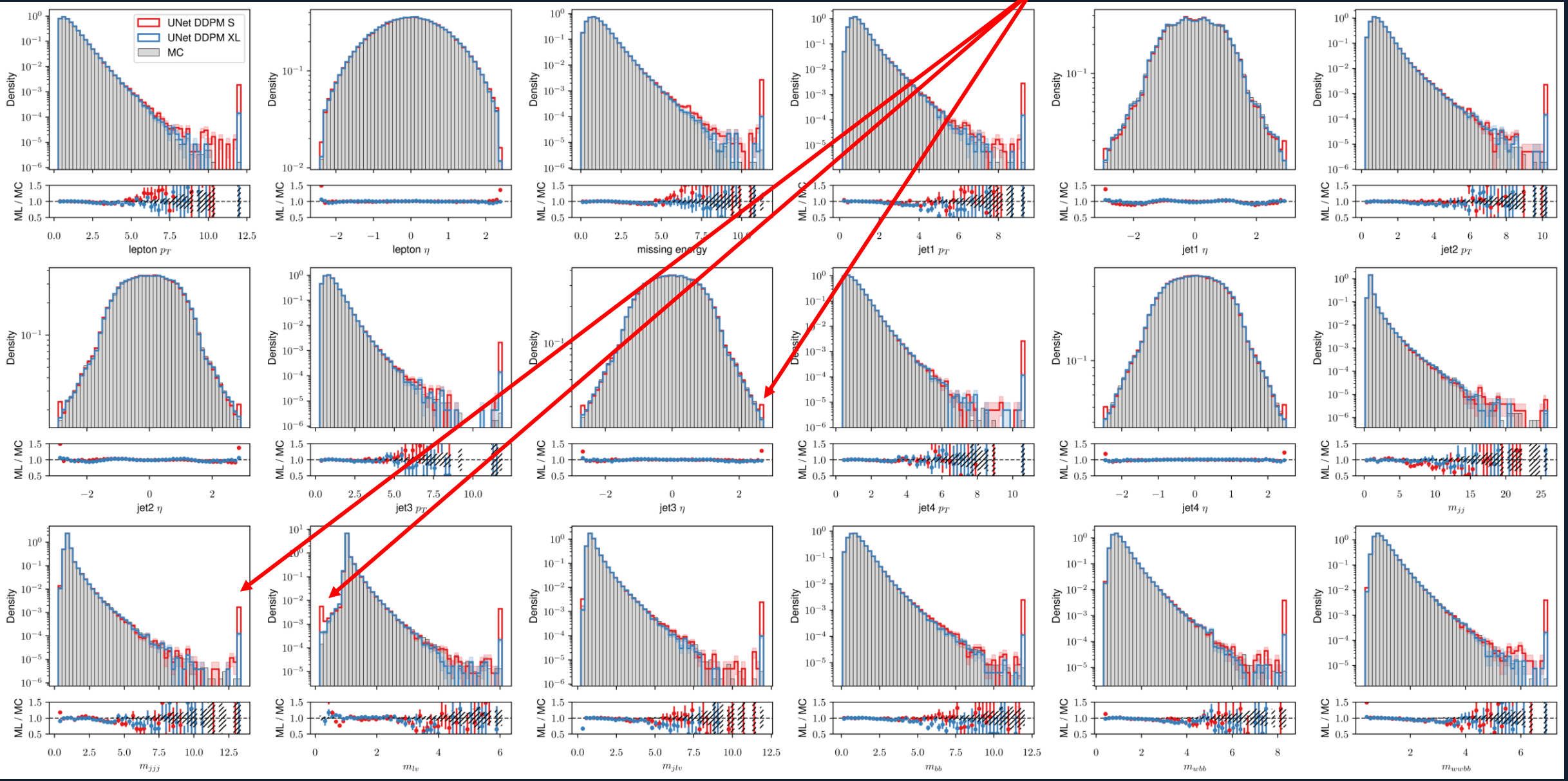
1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

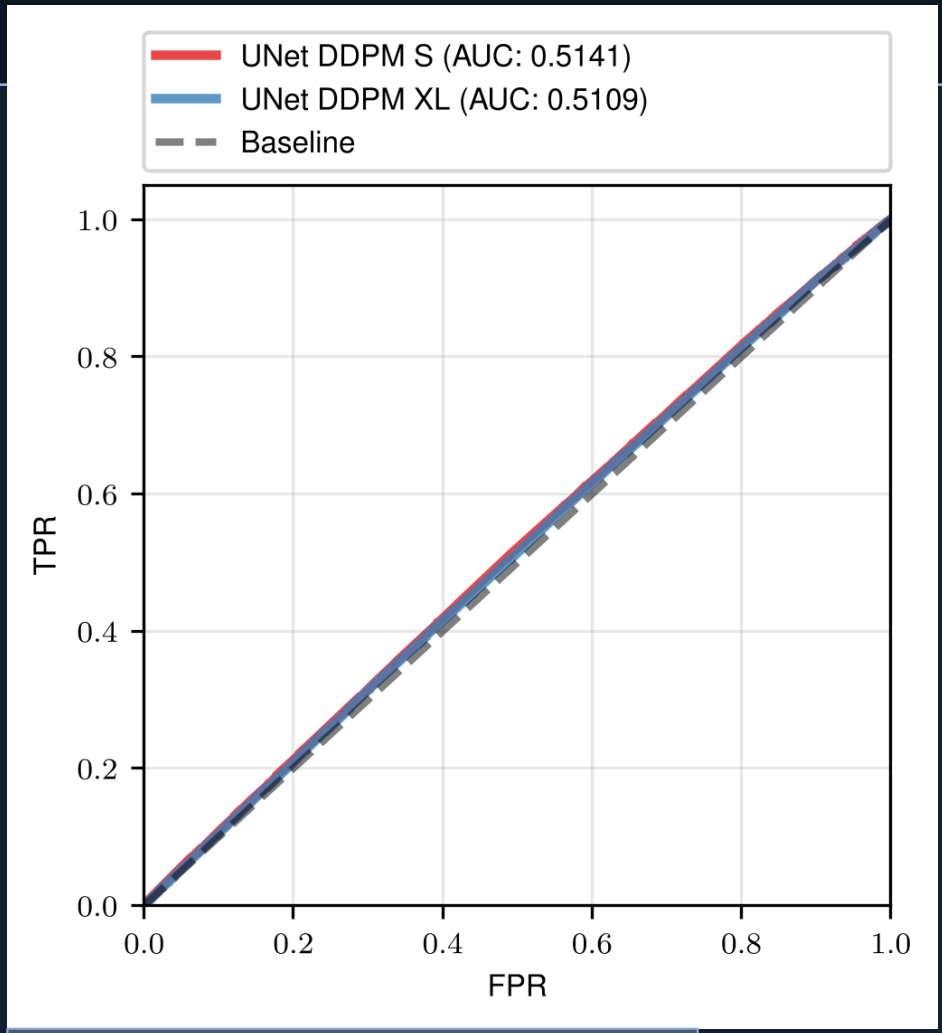
DDPM sampling

After inverting the (rank-Gauss) transformed sampled data, all the outliers are mapped to tails of distributions



C2ST tests

- Tested for 10^6 generated samples for all models
- Data quality probably lowered by large variance for DDPM

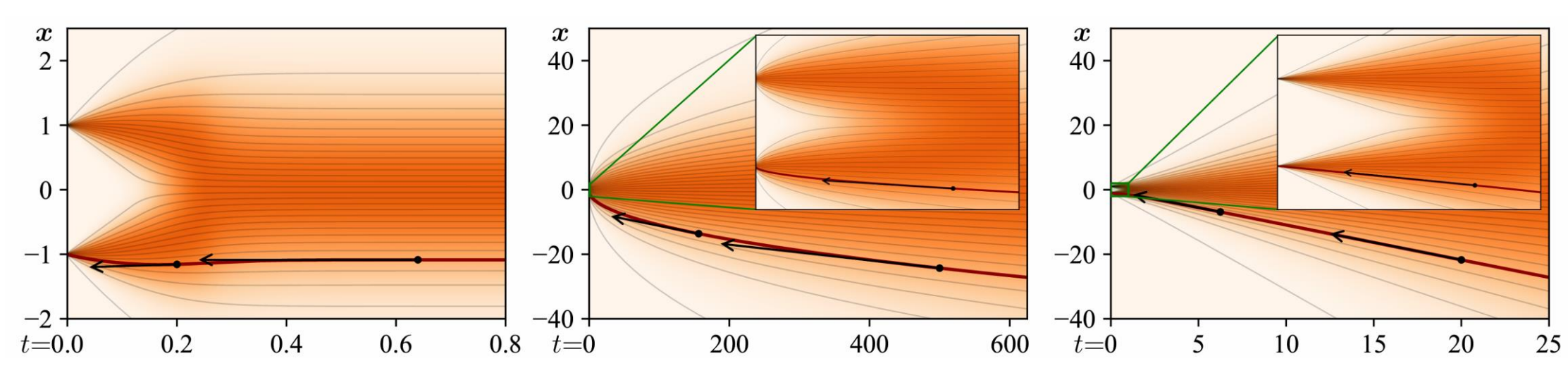


AUC classifier scores for DDPM models

Score based frameworks

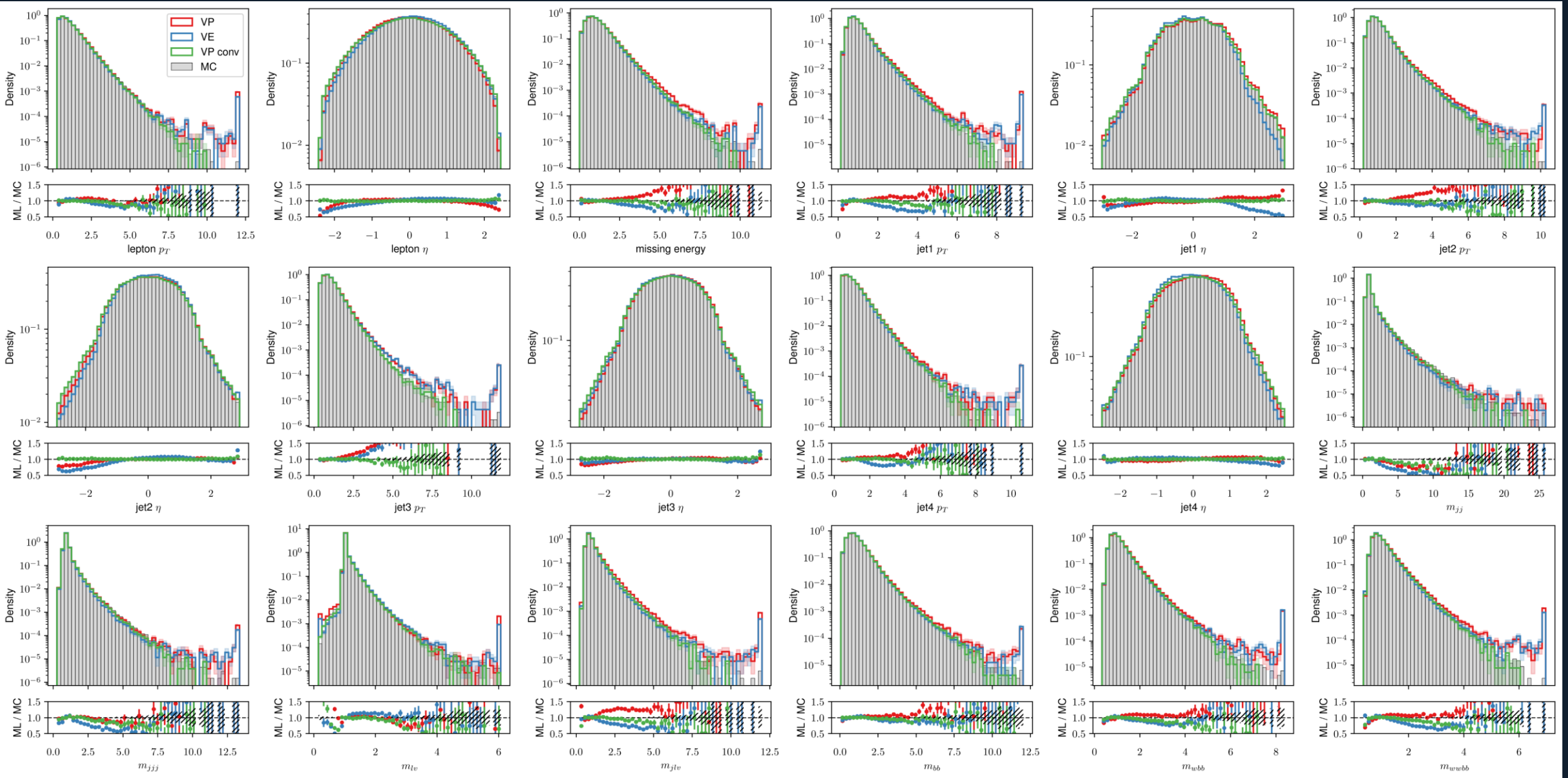
- Variance preserving (VP) – continuous version of DDPM, stable
- Variance exploding (VE) – nonlinear in the low noise region
- EDM – straightest trajectories = better sampling? (in theory)

VP VE EDM



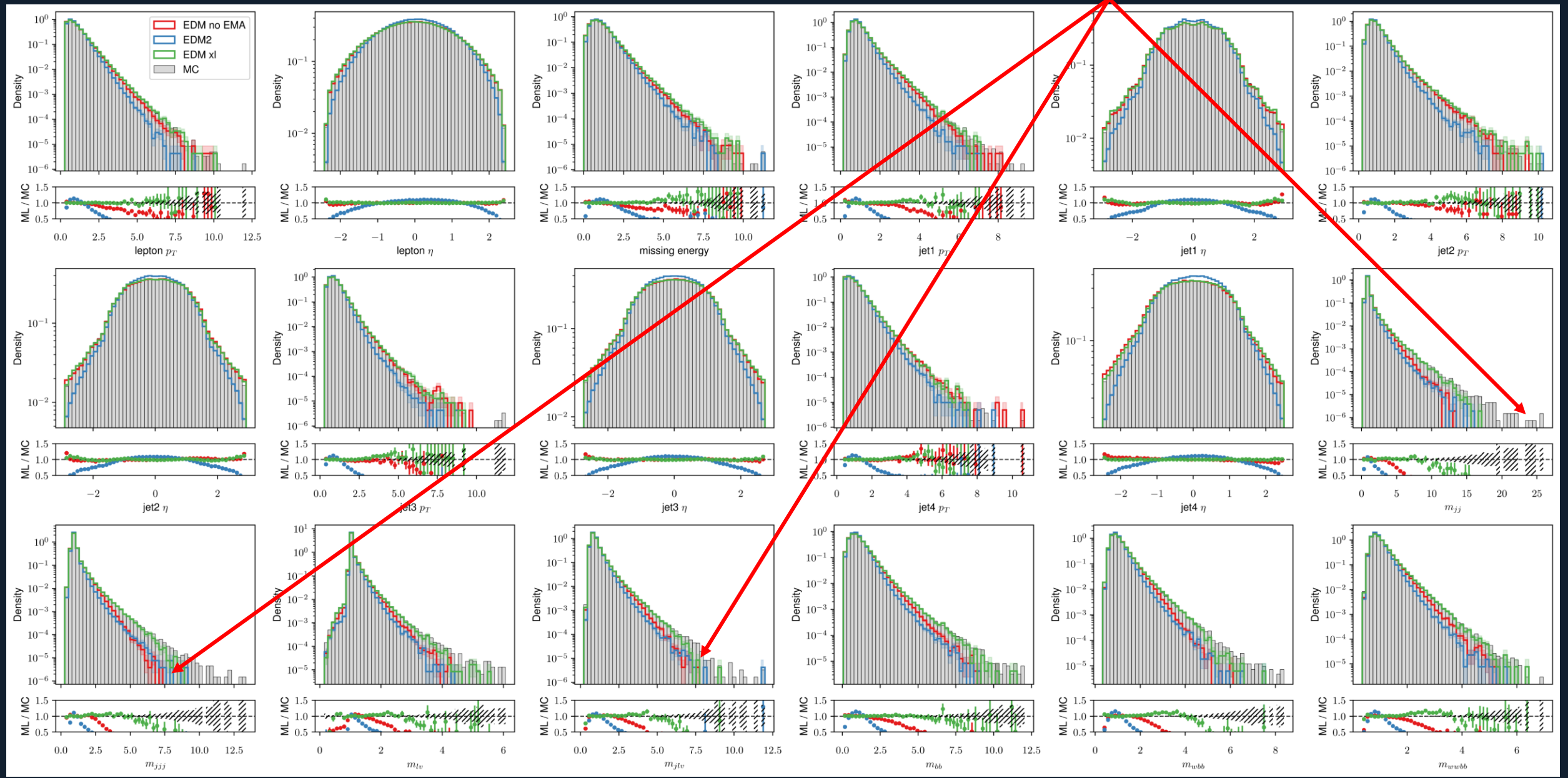
Example trajectories for the 3 variants, starting from two delta functions at $t = 0$.

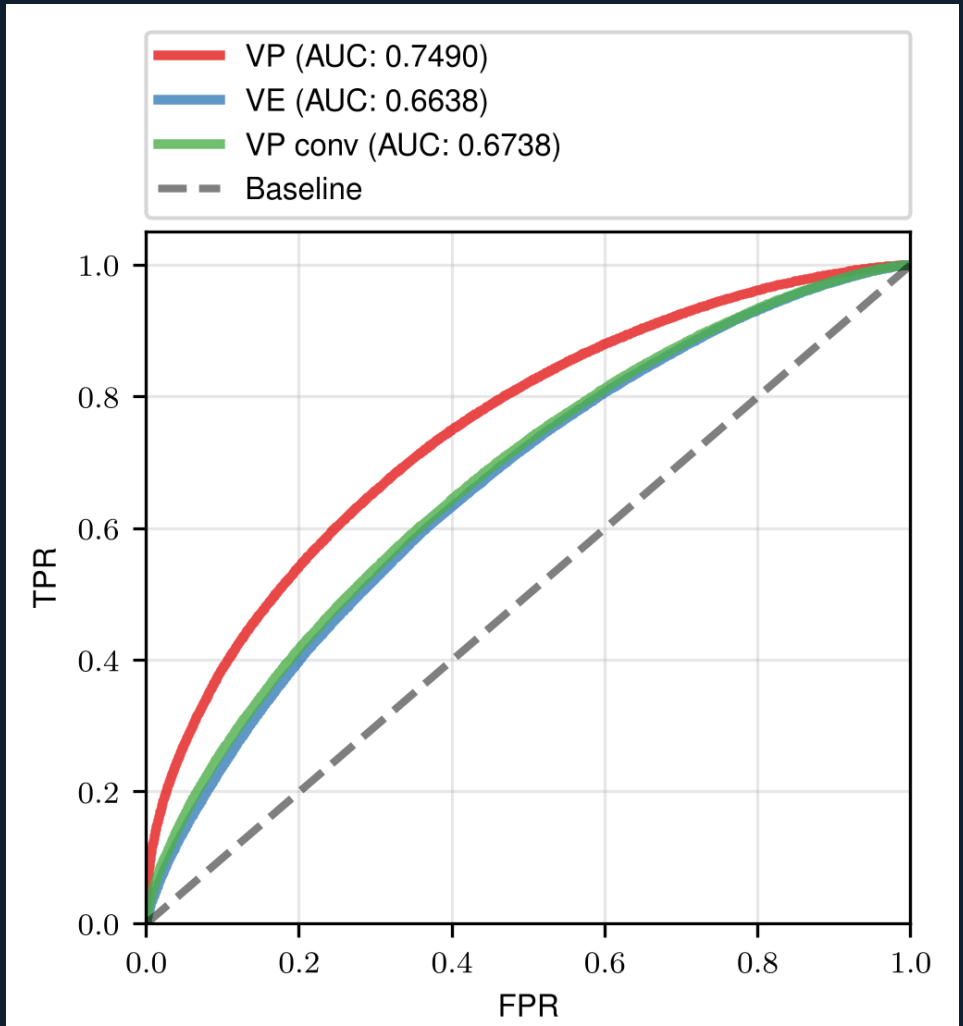
VP, VE, VP (conv. net) samples



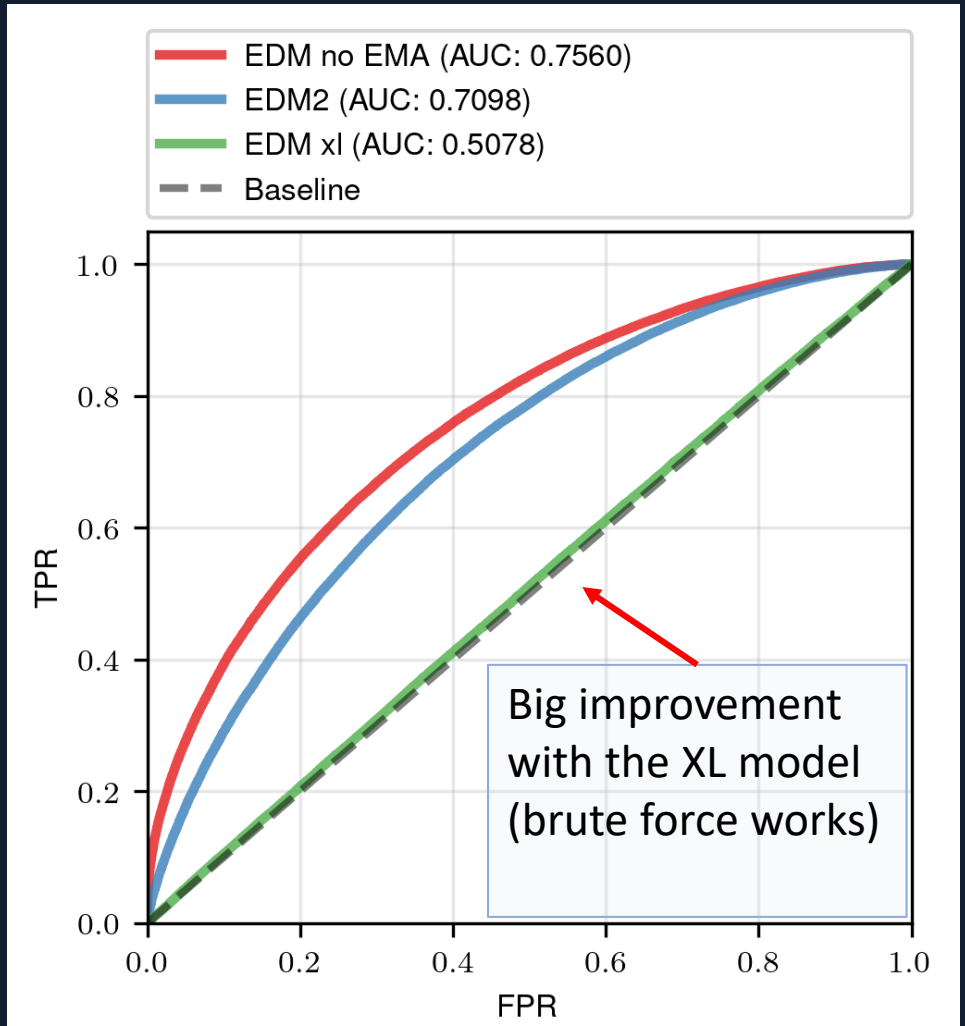
EDM, EDM2 samples

Variance undershooting problems/samples converge to the middle in high level features.





AUC classifier scores for VP, VE and VP conv. models



AUC classifier scores for EDM models

EDM sampling problems

that excessive Langevin-like addition and removal of noise results in gradual loss of detail in the generated images with all datasets and denoiser networks. There is also a drift toward oversaturated colors at very low and high noise levels. We suspect that practical denoisers induce a slightly non-conservative vector field in Eq. 3, violating the premises of Langevin diffusion and causing these detrimental effects. Notably, our experiments with analytical denoisers (such as the one in Figure 1b)

$$\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x};\sigma) = \big(D(\boldsymbol{x};\sigma) - \boldsymbol{x}\big)/\sigma^2$$

If the degradation is caused by flaws in $D_\theta(\boldsymbol{x};\sigma)$, they can only be remedied using heuristic means during sampling. We address the drift toward oversaturated colors by only enabling stochasticity within a specific range of noise levels $t_i \in [S_{\text{tmin}}, S_{\text{tmax}}]$. For these noise levels, we define $\gamma_i =$

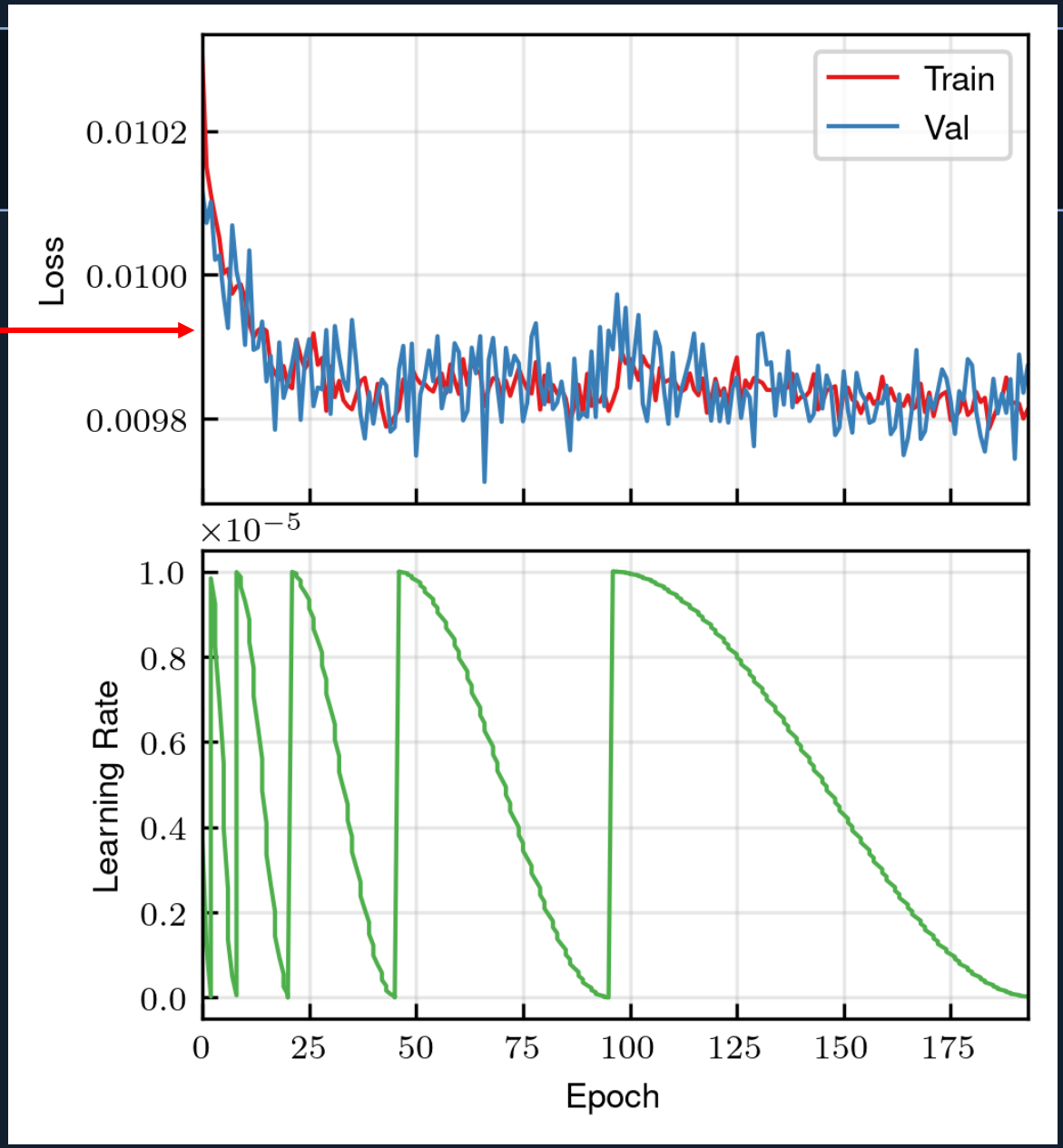
These have been „optimized“, but low variance problems remain after training.

EDM training

Starting loss already small, then it barely improves.

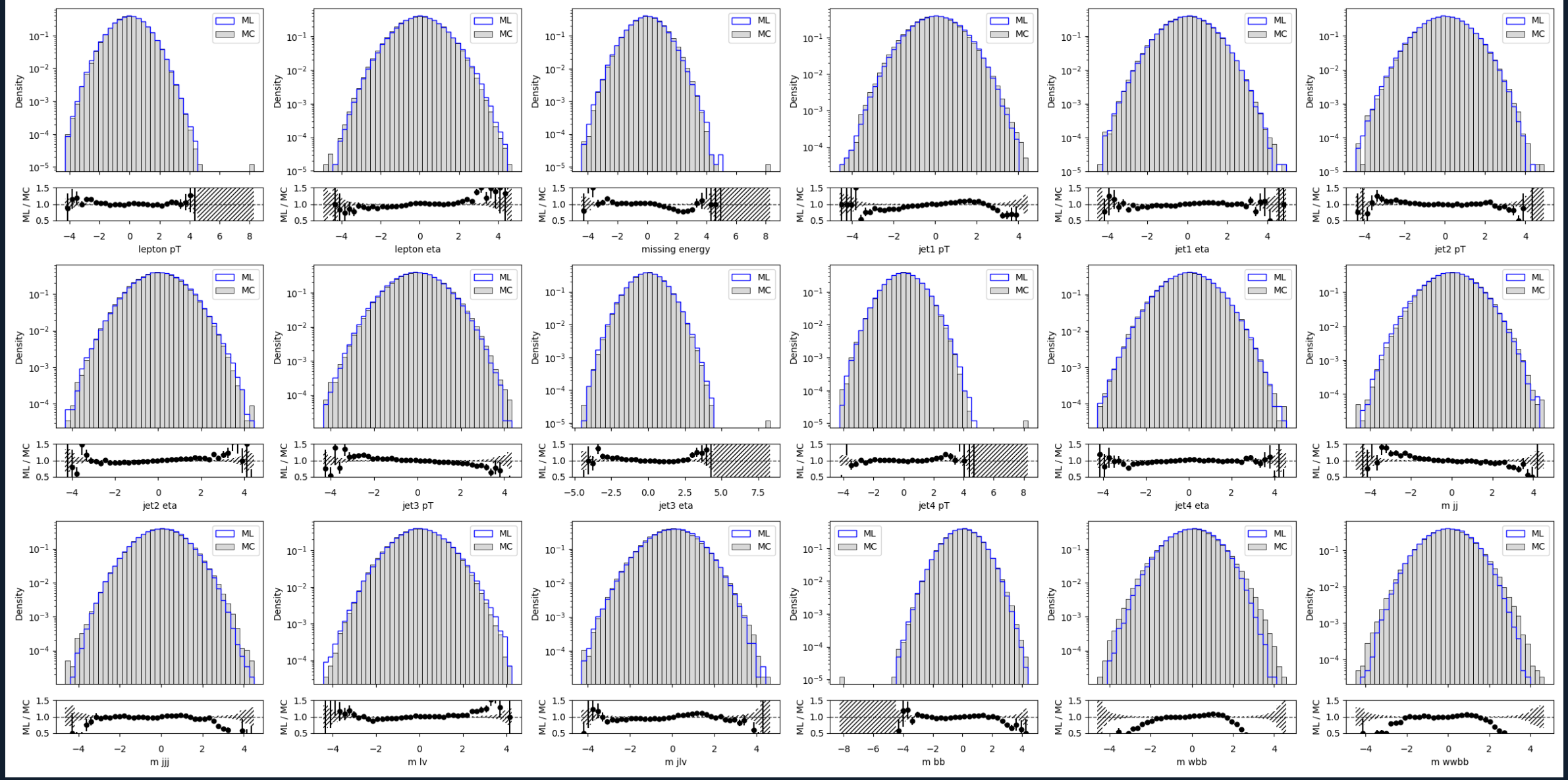
Noisy even at low learning rates (1E-5)

Cosine annealing schedule with warm restarts and increasing period worked best.



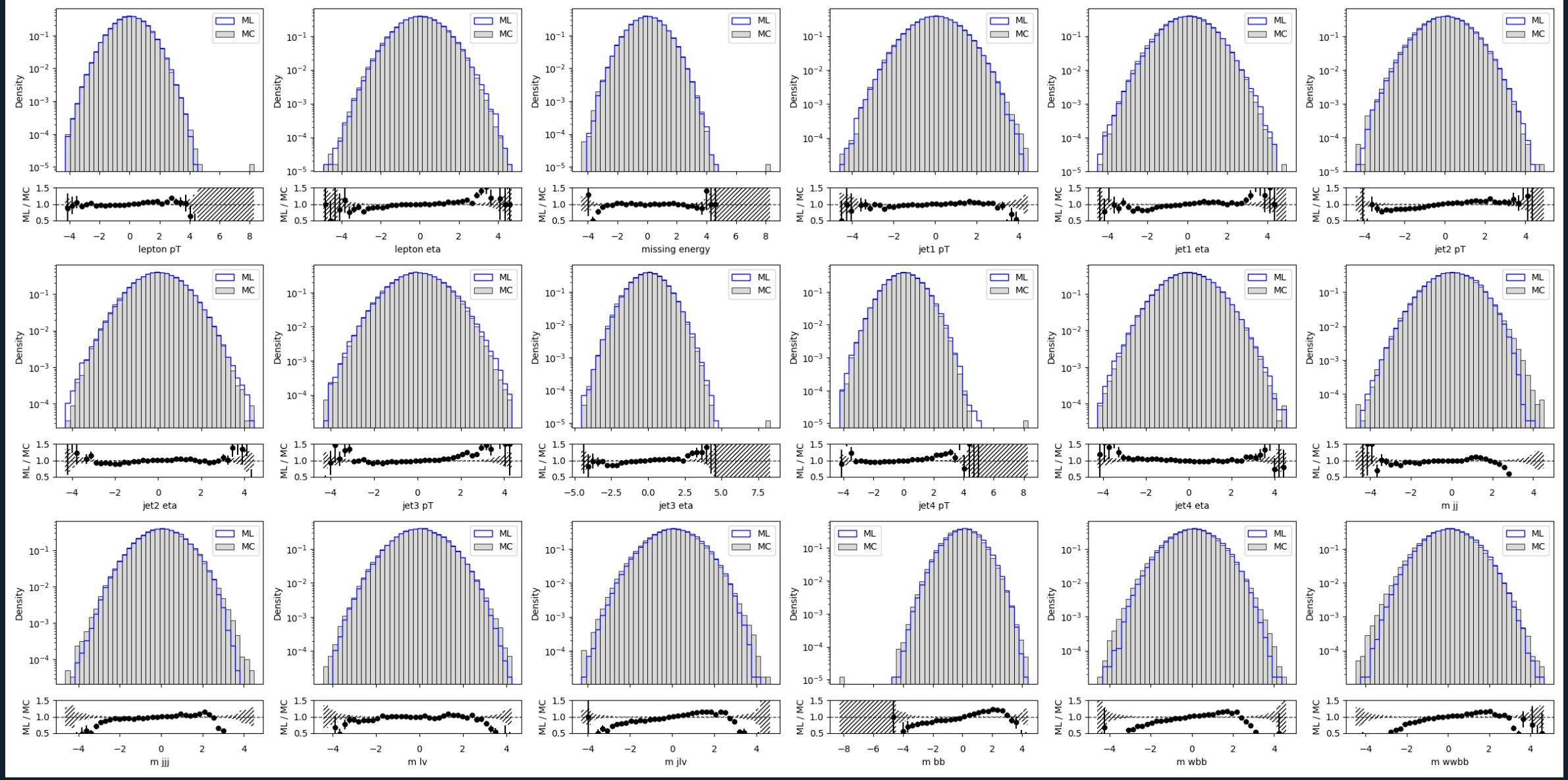
Training samples snapshots for EDM

Epoch 0/196



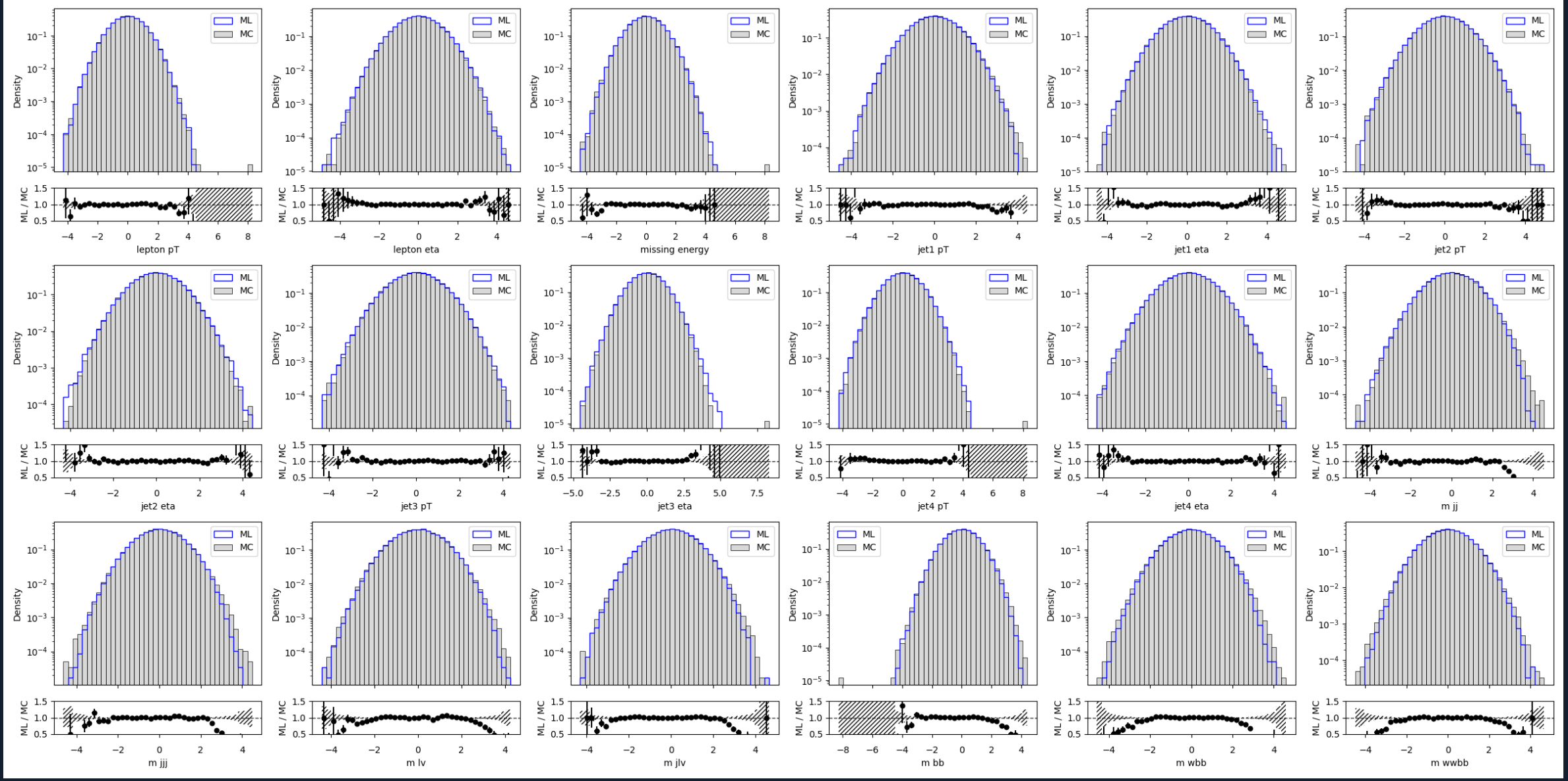
Training samples snapshots for EDM

Epoch 93/196



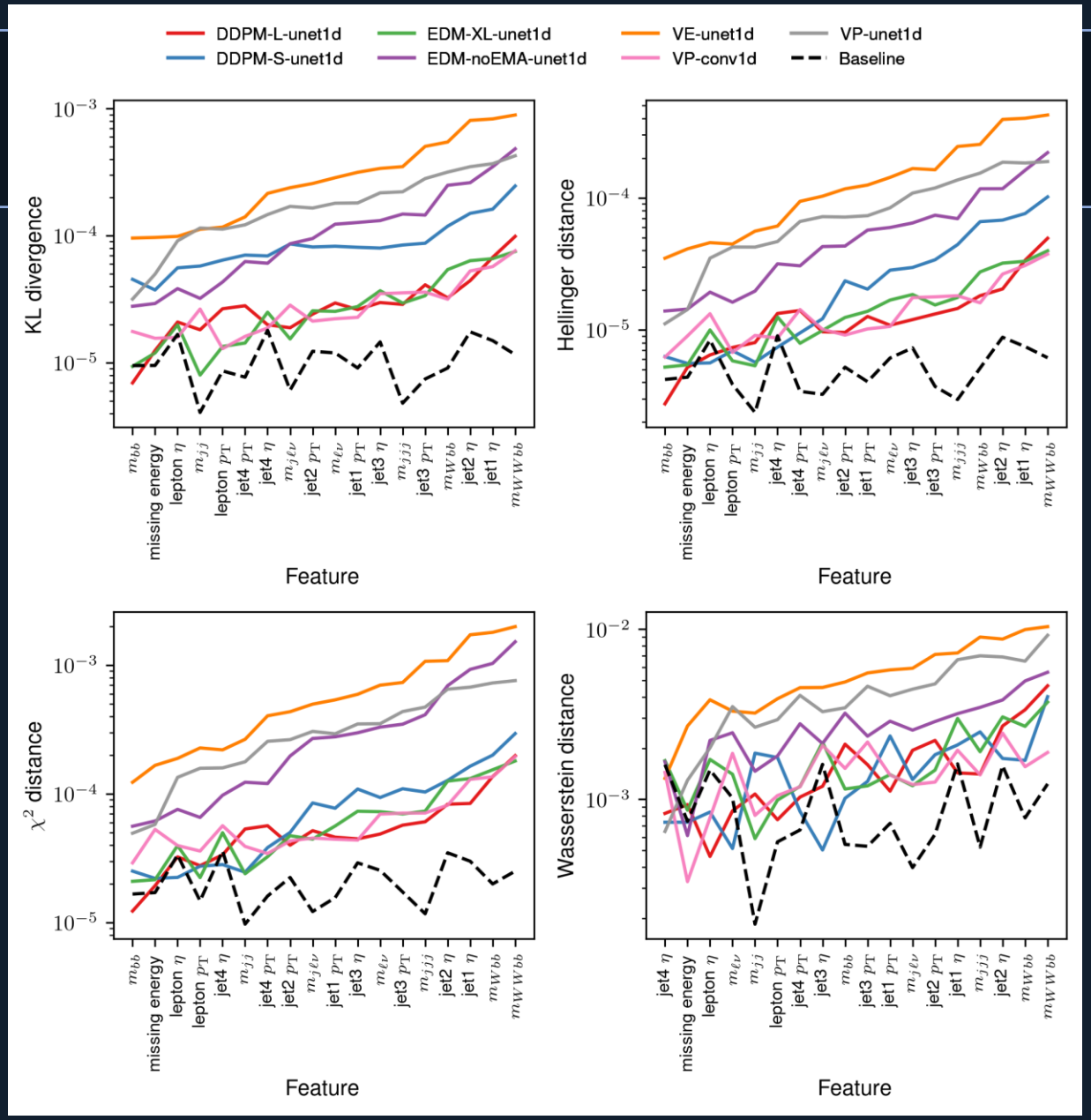
Training samples snapshots for EDM

Epoch 192/196



Model distances

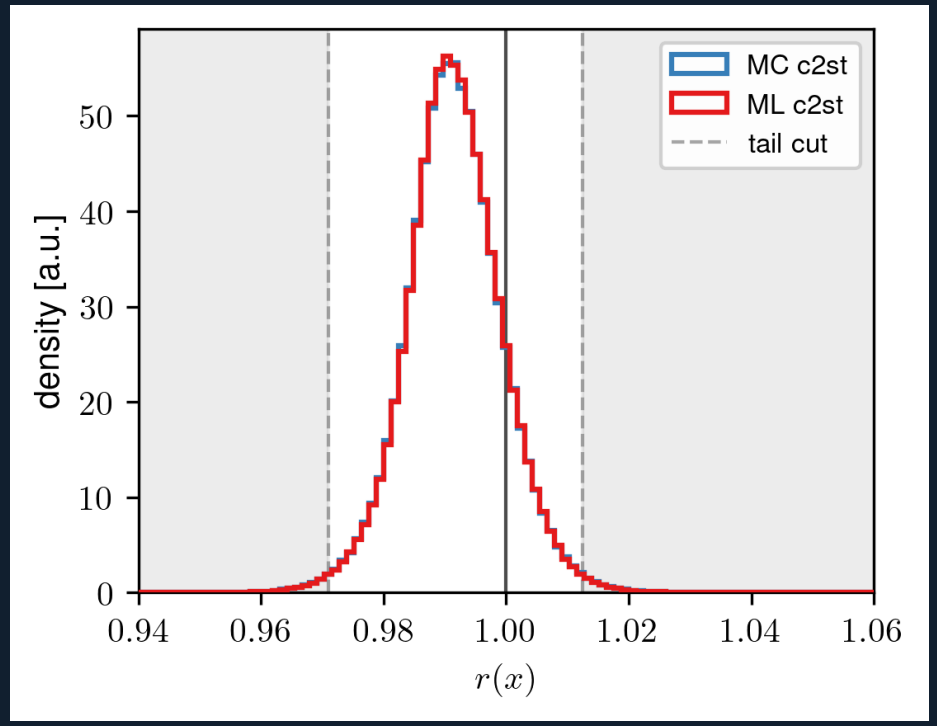
- Best performers: EDM xl, VP conv. and DDPM
- VP conv. underperforms on the C2ST test despite good distance results



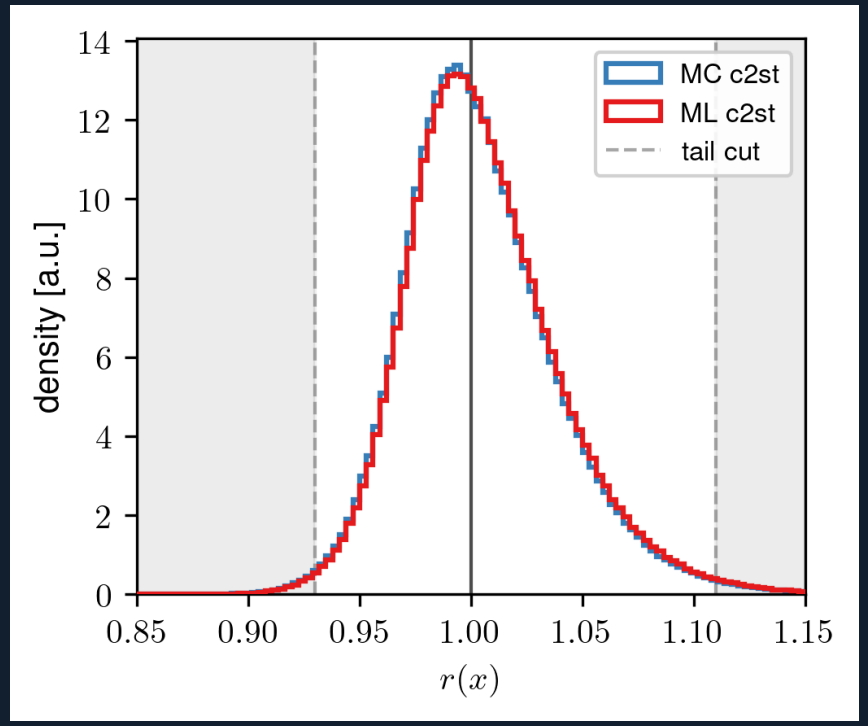
Ratio tests

C2ST Classifier output

- $r(x)=P_{ML}(x) / P_{MC}(x)$ estimated as $r(x)\approx\exp(\sigma^{-1}[f(\mathbf{x};\theta)])$
- ~1.5 % of data in the tails



DDPM

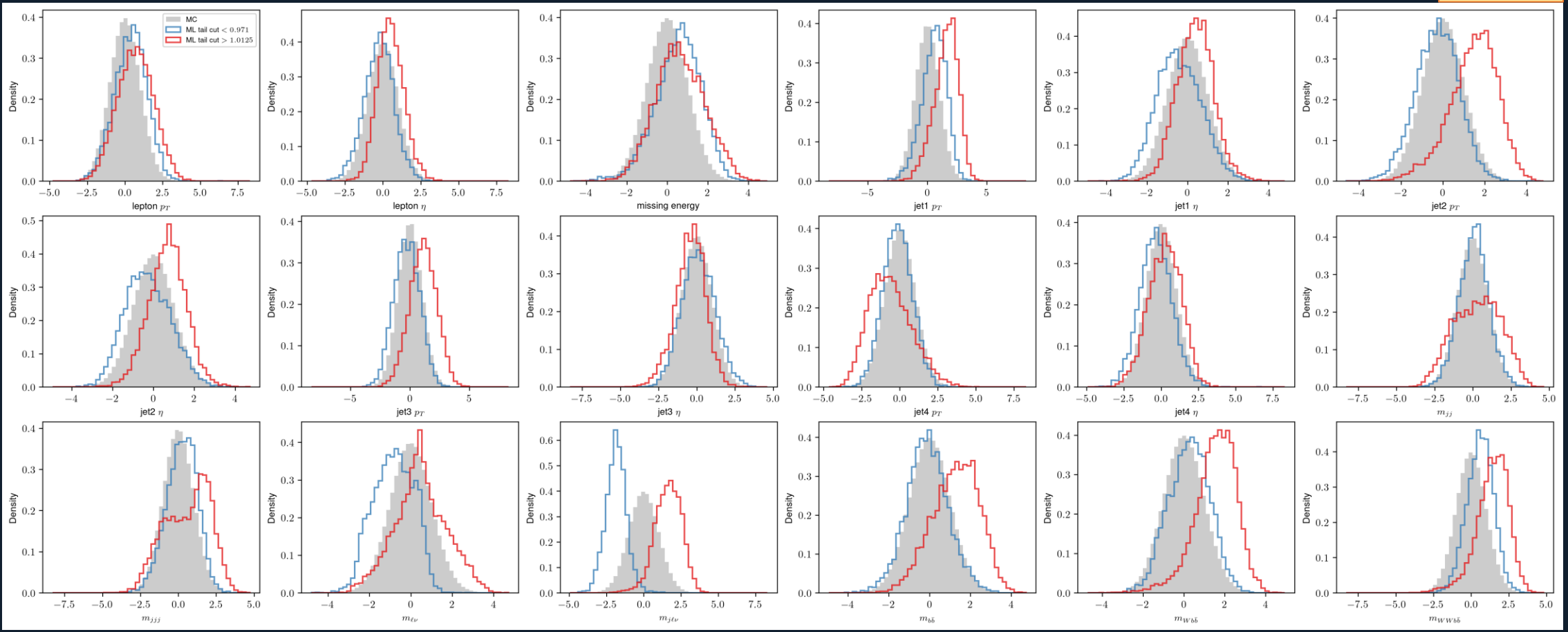


EDM (xl model)

Distributions after the cut

- EDM struggles with high level features, low varince problem

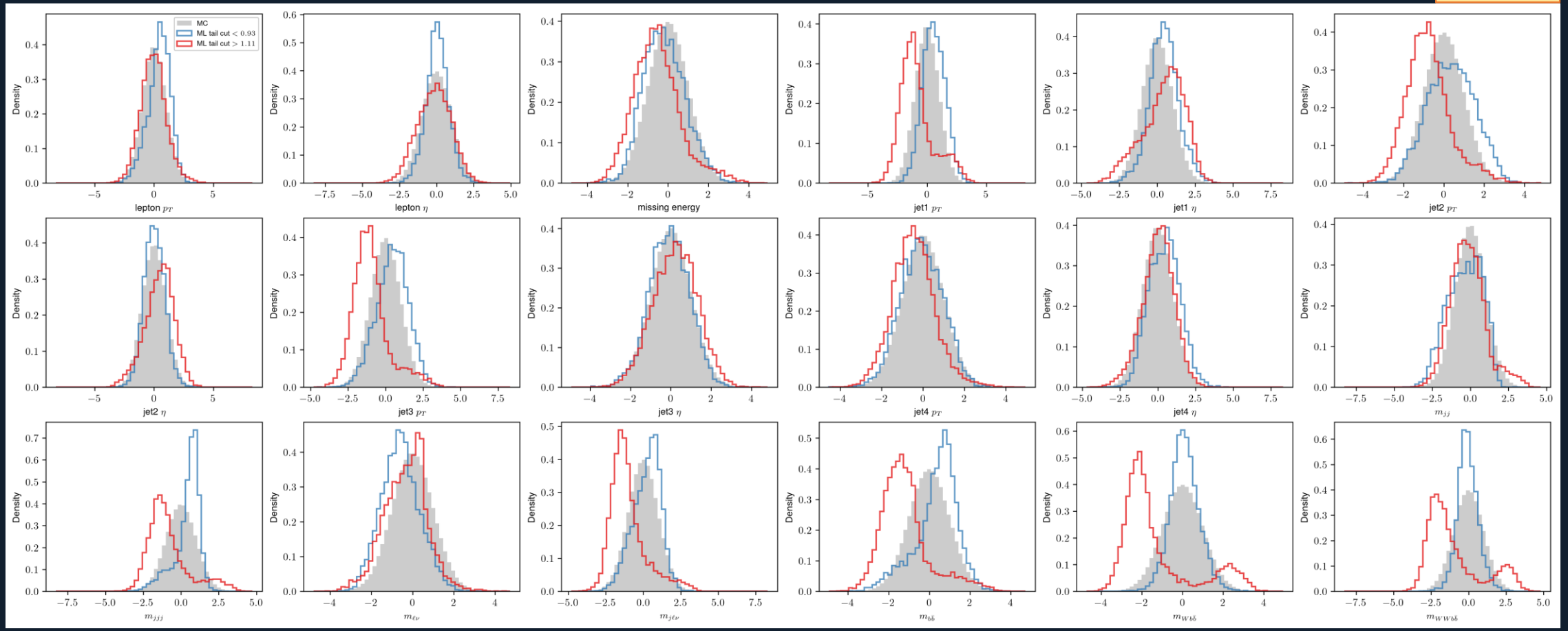
DDPM



Distributions after the cut

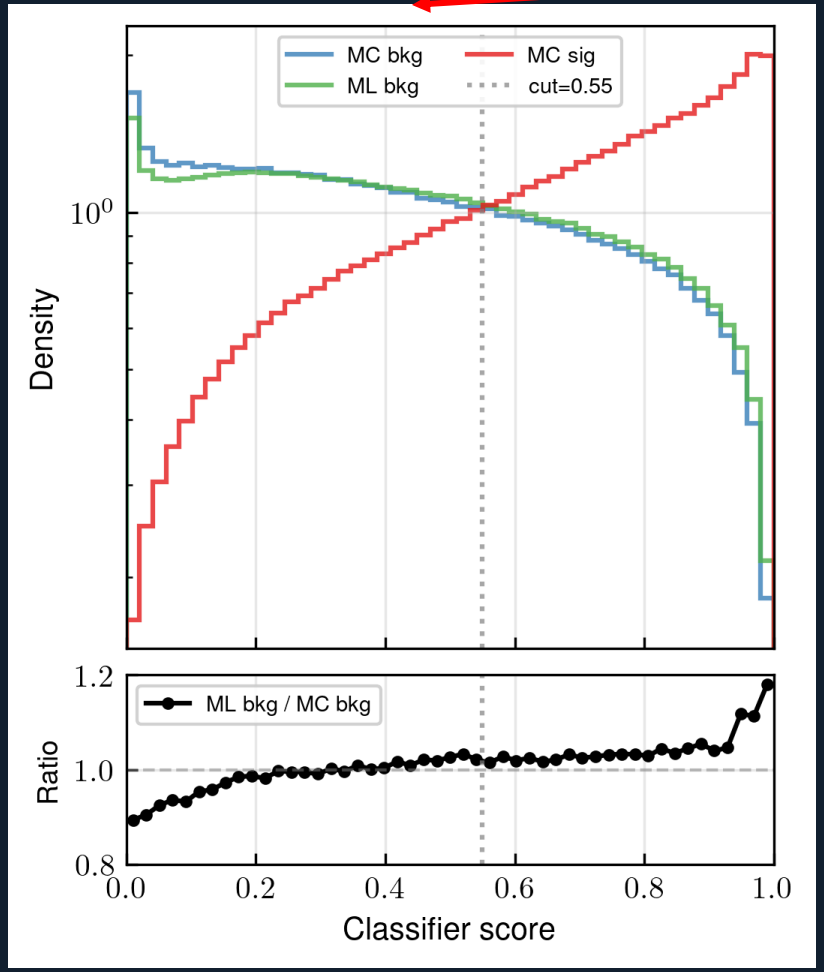
- EDM struggles with high level features, low varince problem

EDM

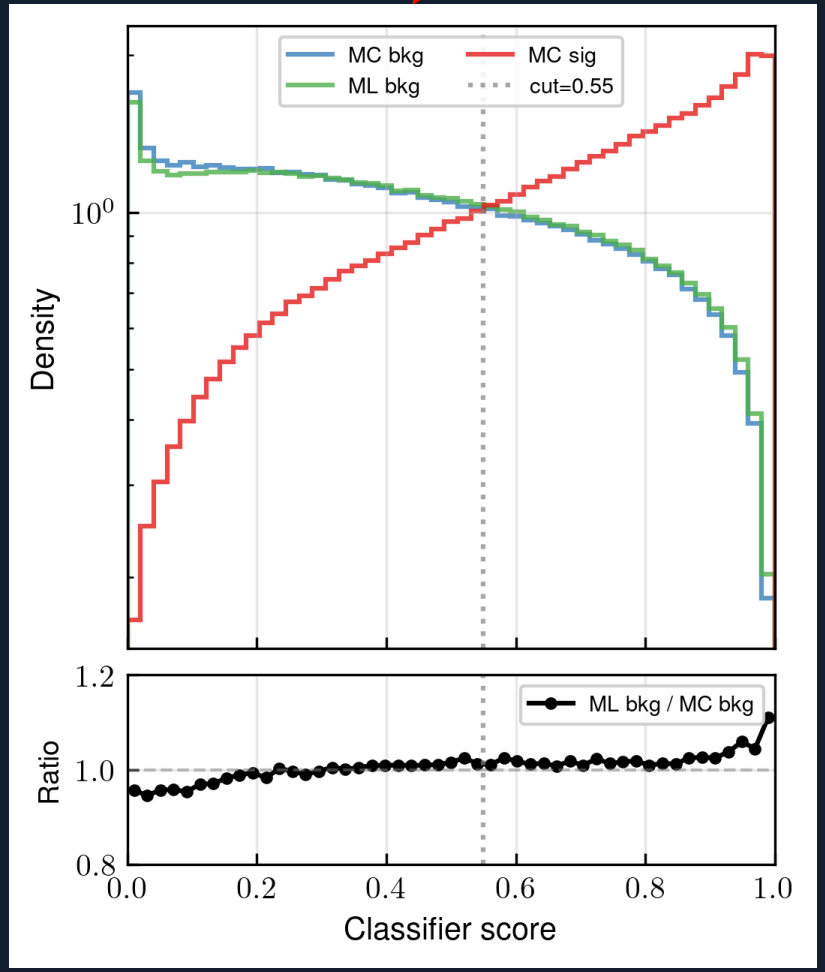


Sig.-Bkg. ratios

Sig.-bkg. classifier outputs for MC sig., MC bkg. and ML bkg. (~2.6 mil. events each)



DDPM

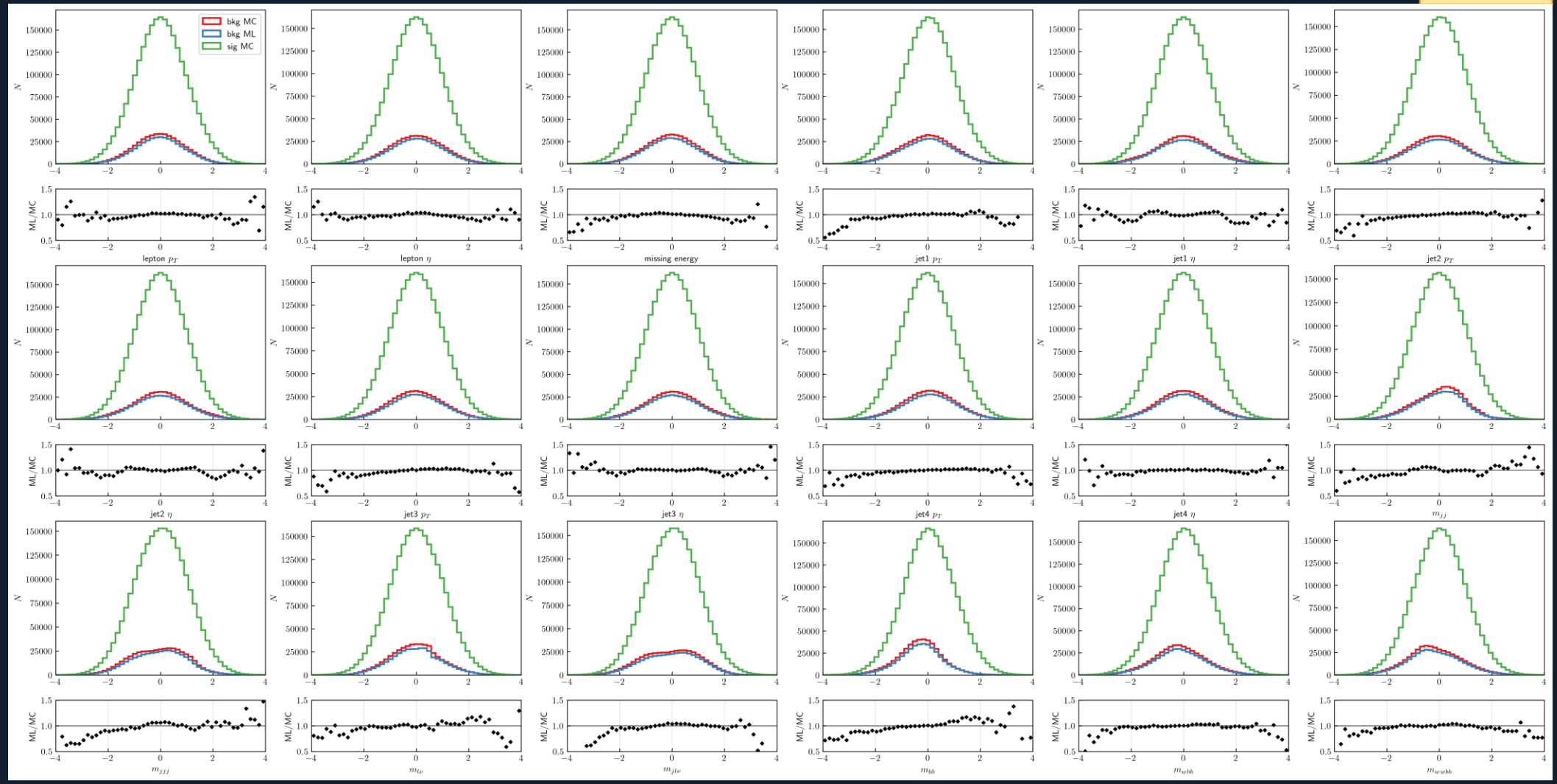


EDM (xl model)

Distributions after the cut

DDPM

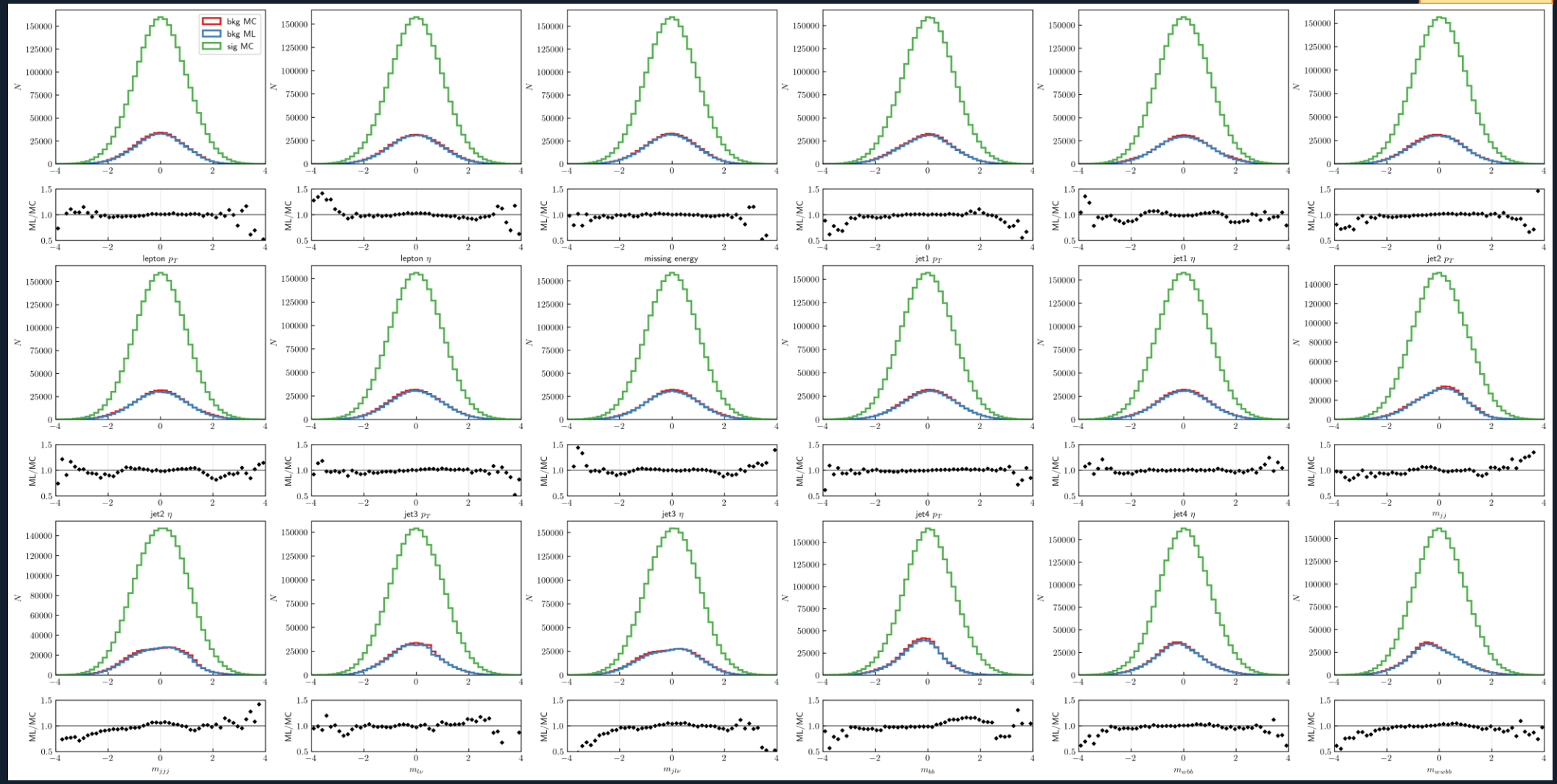
Distributions after the cut on the classifier, at score of 0.55
+
ratios of bkg. ML/MC densities



Distributions after the cut

EDM

Distributions after the cut on the classifier, at score of 0.55
+
ratios of bkg. ML/MC densities



Todo list

i. EDM framework improvements: reweighing data, better networks?
ii. New architectures → small data dim. = problems with conv. networks
iii. Consistency models for fast sampling
iv. Training parameter and sampling ODE solver parameter optimization
v. Power function EMA parameter optimization/quality quantification
vi. Autoguidance?
vii. Better preconditioning for DDPM

Conclusions

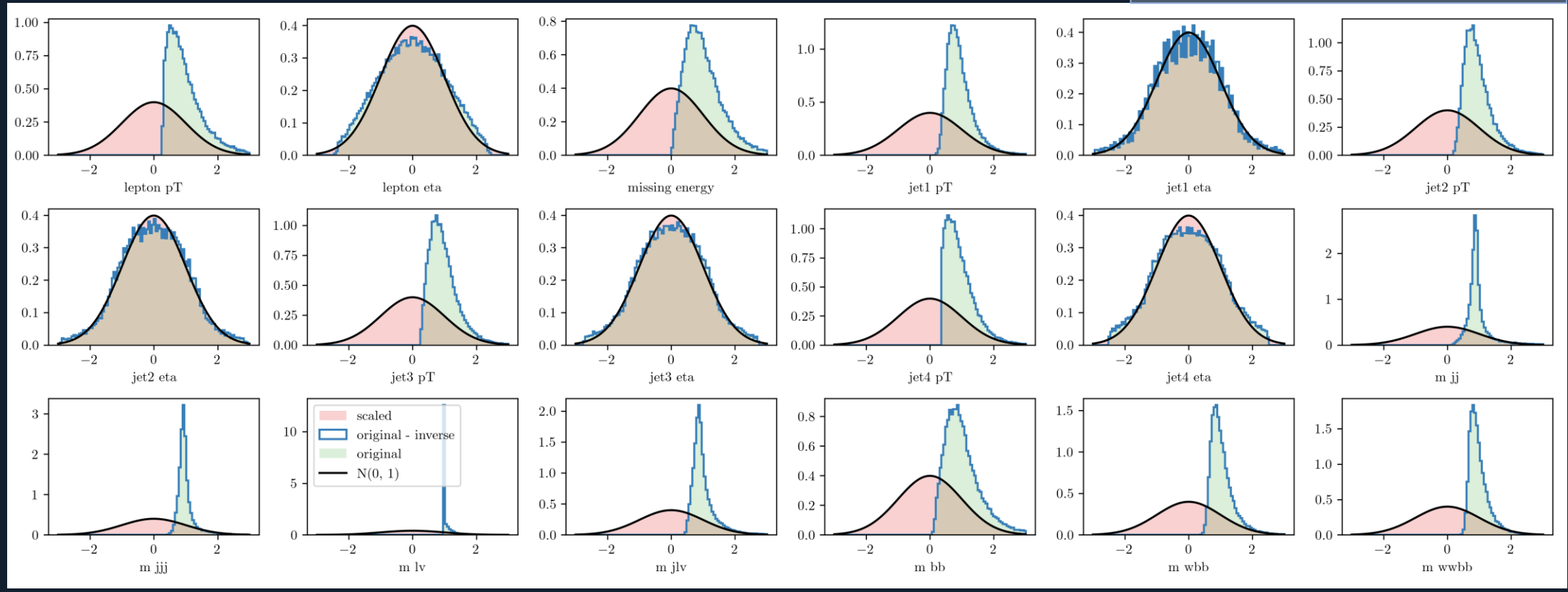
- Promising frameworks: standard stohastic DDPM, EDM framework?
- Score/SDE networks: ODE sampling = less steps, faster but lowers samples‘ variance / quality

Supplementary info

- Before training the data is transformed using the **Rank → Gaussian scaler**

1. Rank transform ~ ECDF/data sort and scale from -1 to 1
2. Apply *invErf*(scaled data) to get a normal distribution

Data, transfromed data, normal distribution (black) and inverse transform sanity checks.

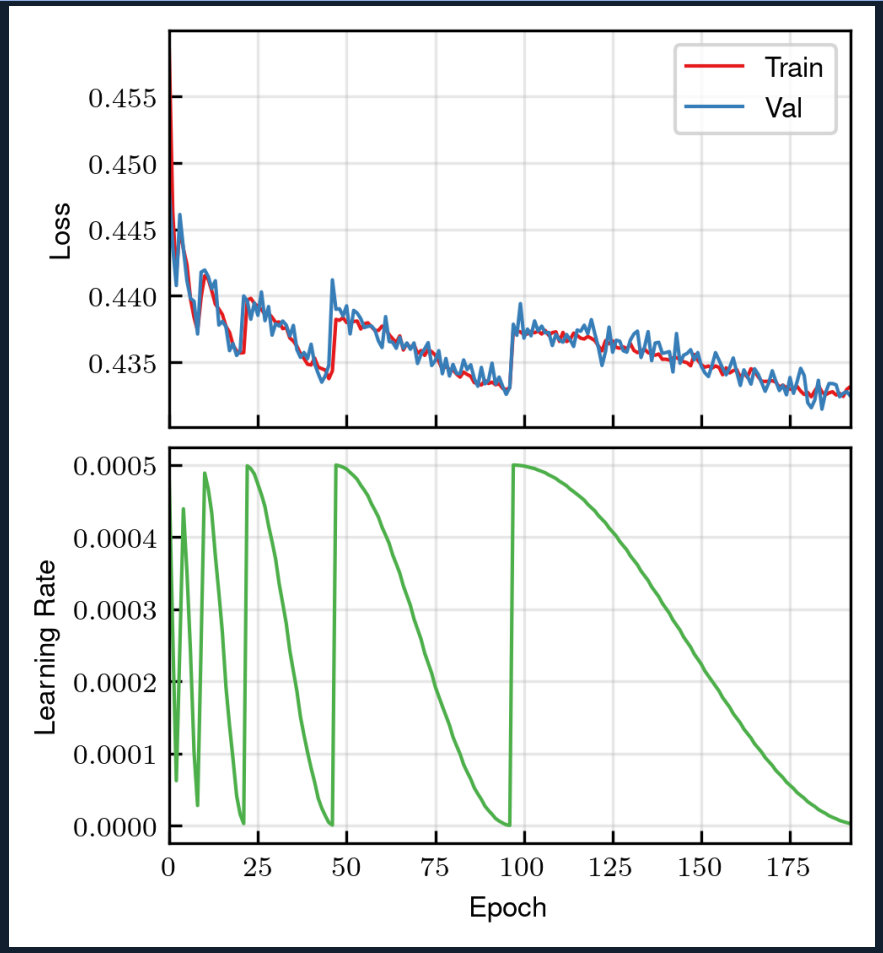


Architectures + weight calculations

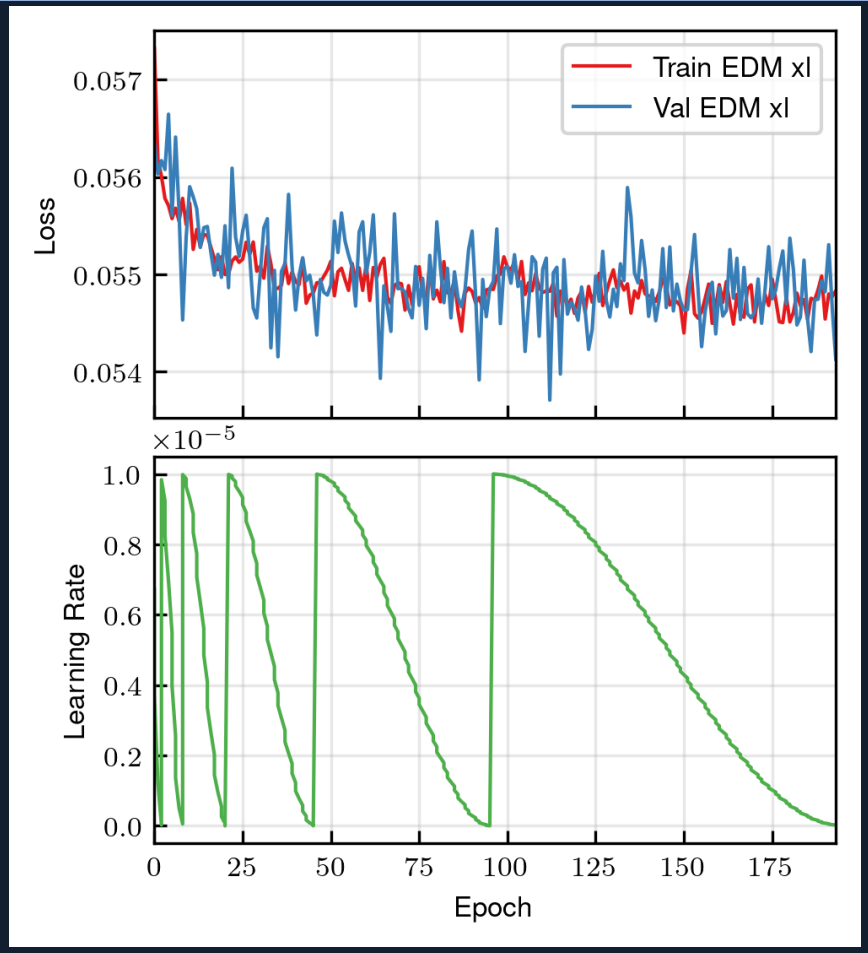
- For most models an MLP U-net was used
- Convolutional U-nets have prolems with small data dimesnionality, learning slower in most cases and not as successful
- Standard sinusoidal time embedding

- Weights saved using a (power funftion) exponential moving average (pfEMA): see Karras EDM2 paper, pg. 5 for details
- Parameters to be optimized, qulatitative analysis (probably) needed

Training/validation loss



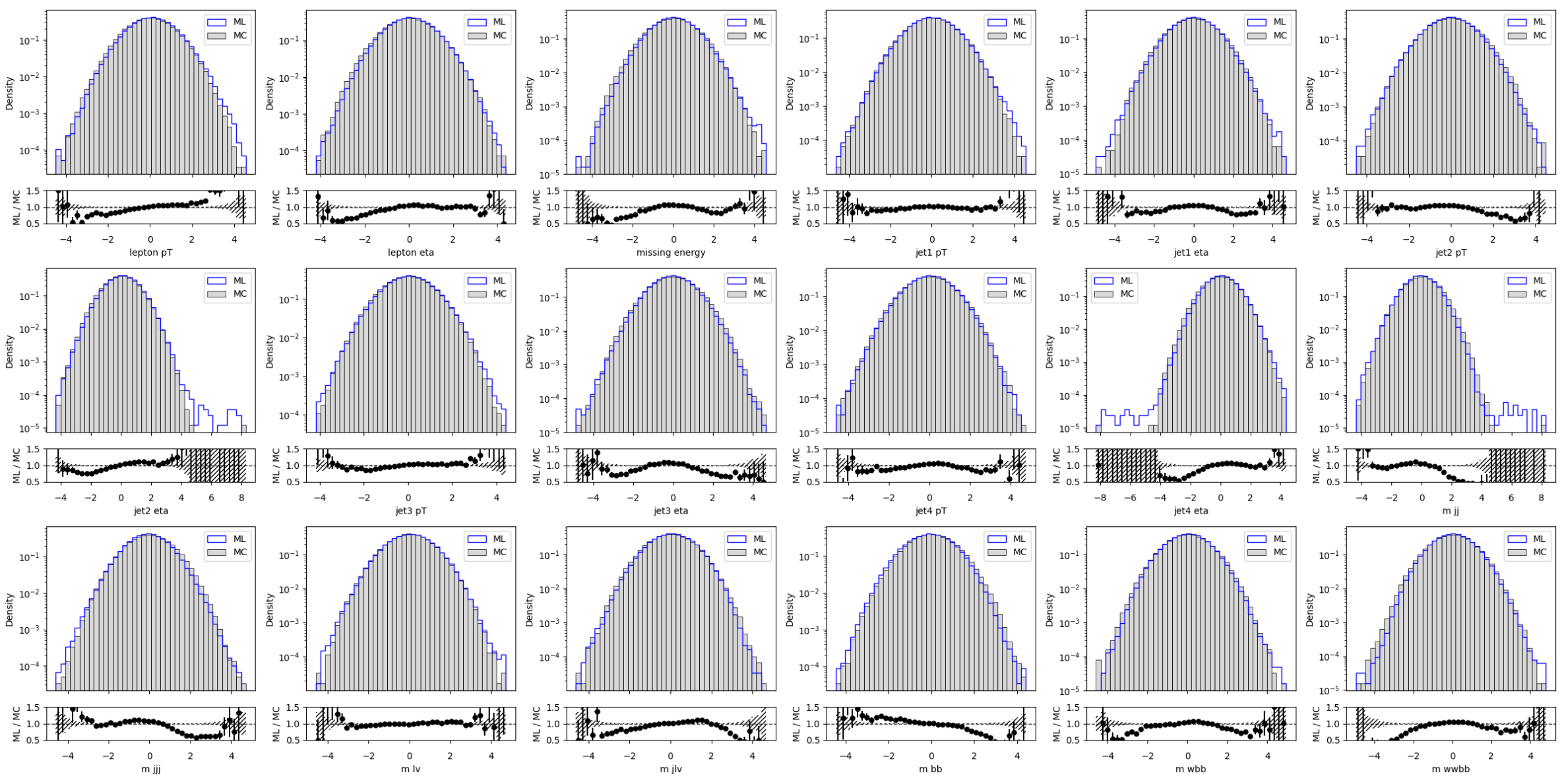
DDPM



EDM (xl model)

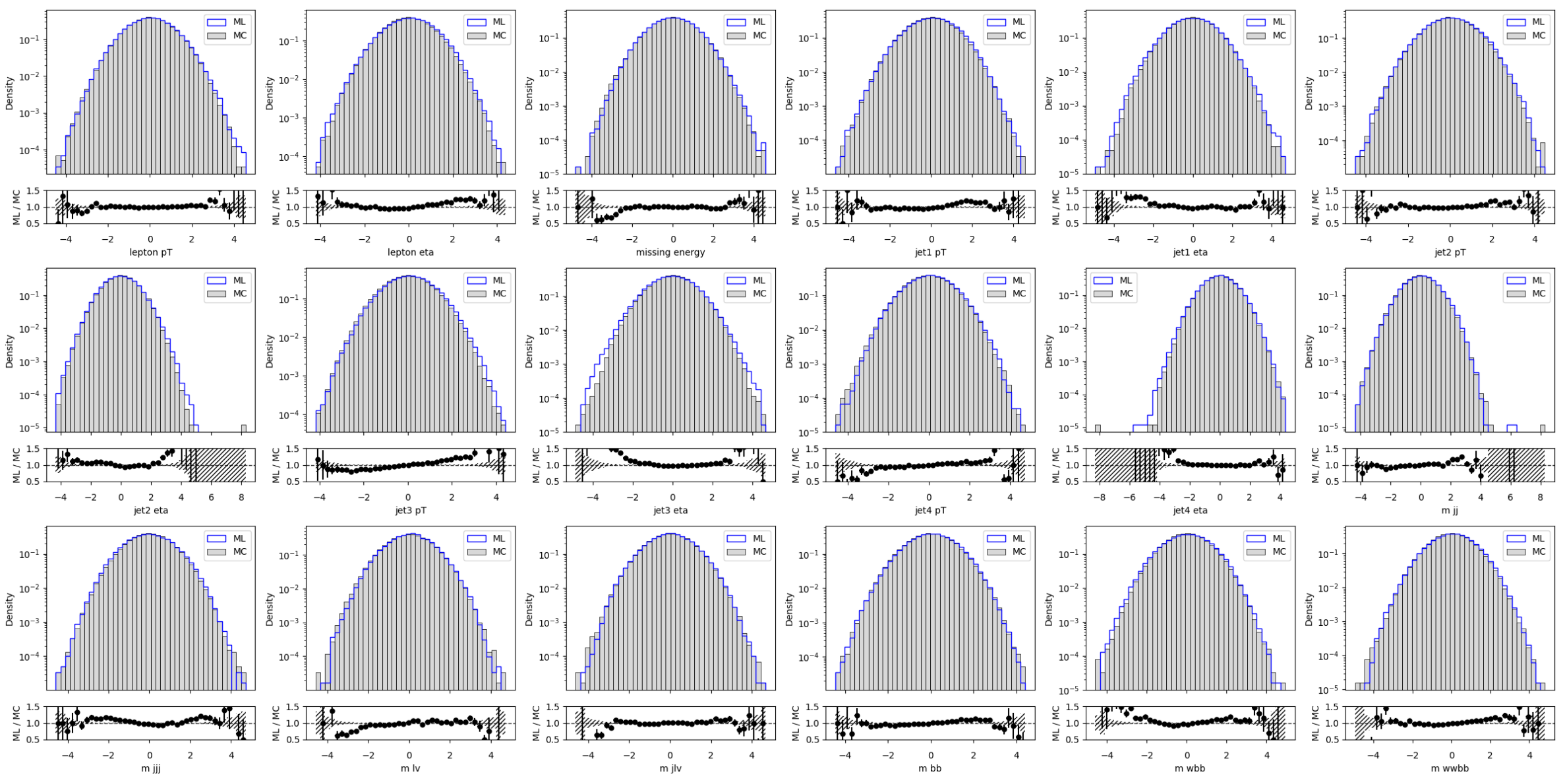
Training samples snapshots for DDPM

Epoch 0/190



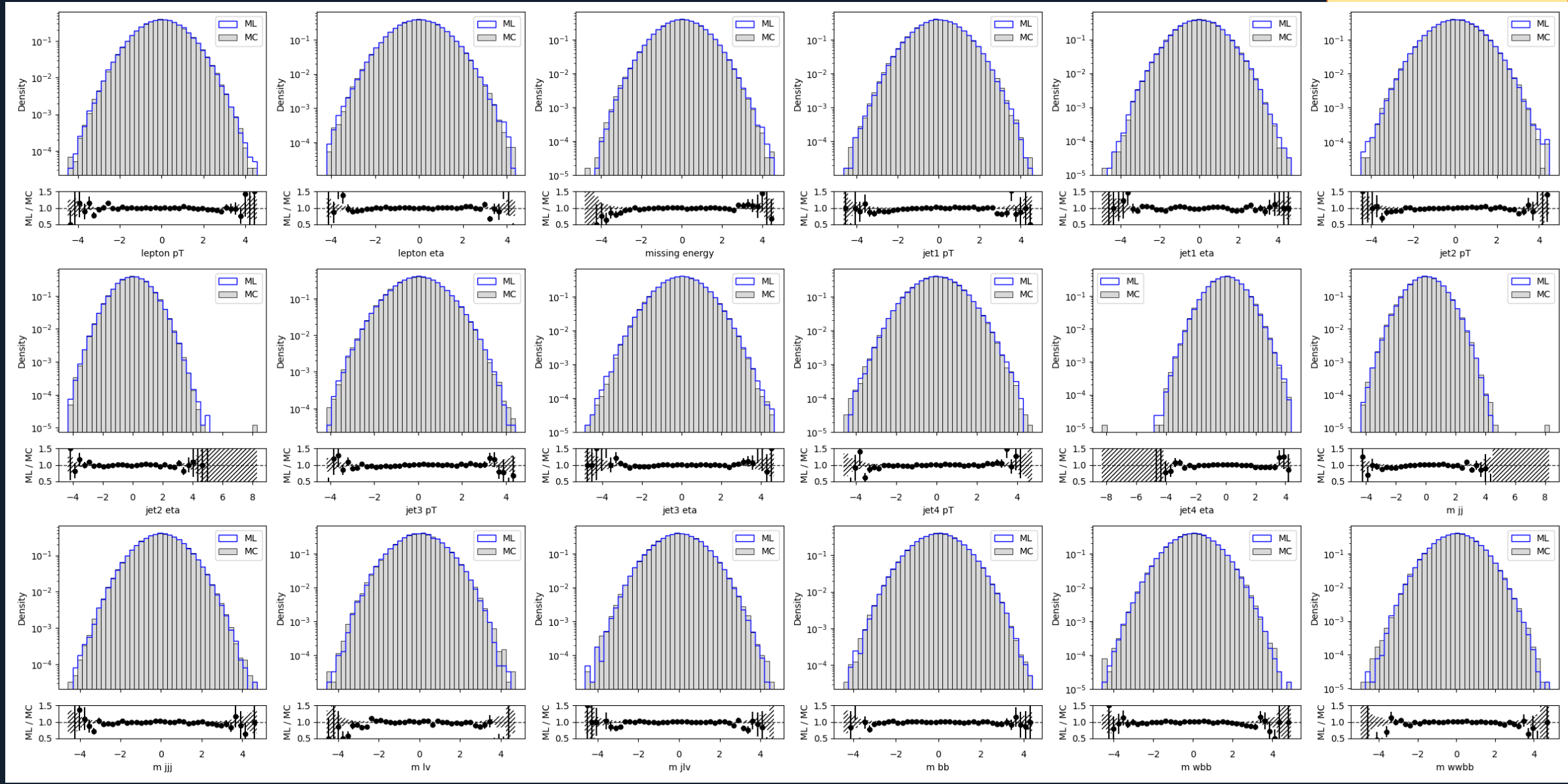
Training samples snapshots for DDPM

Epoch 114/190



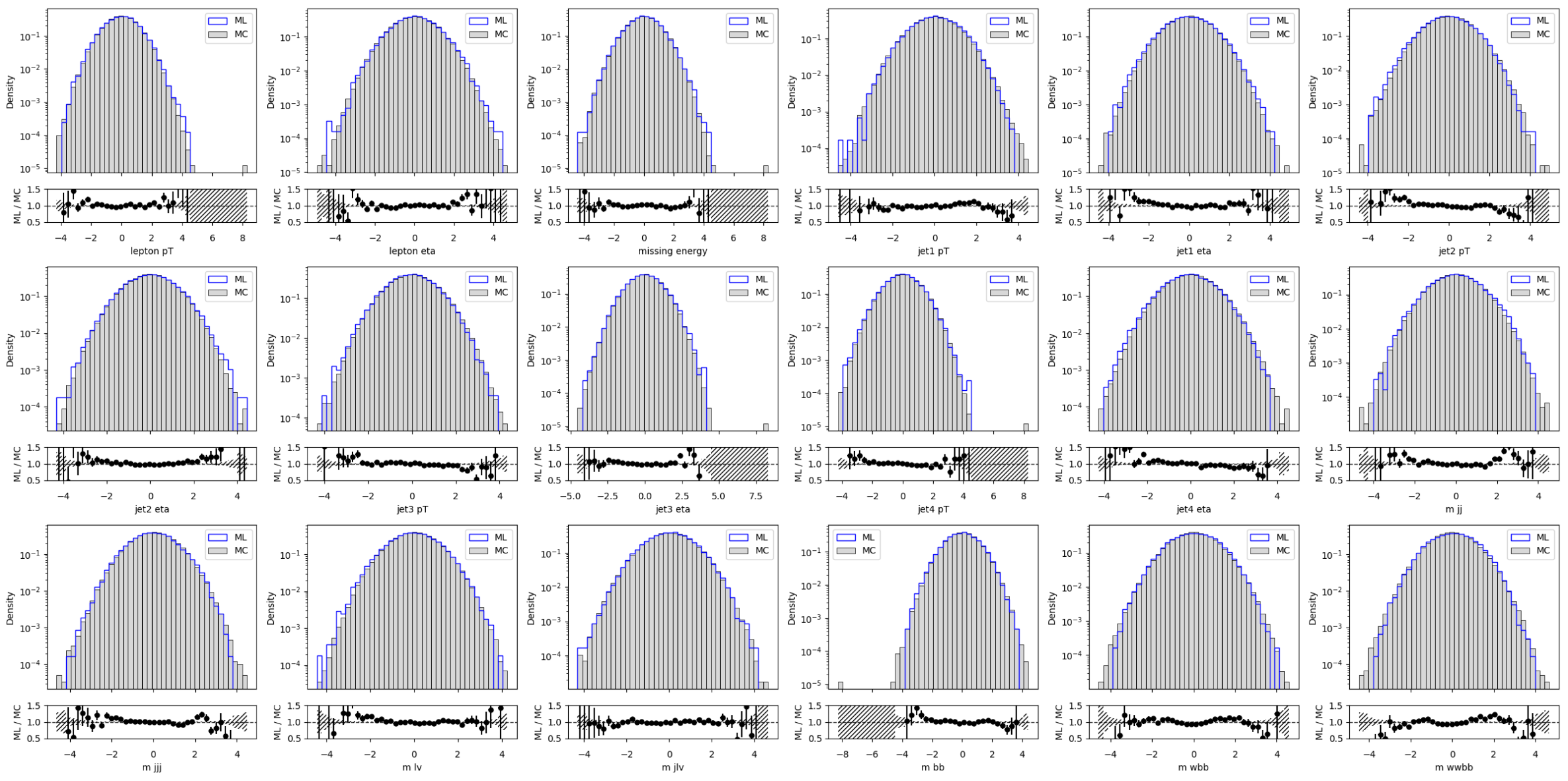
Training samples snapshots for DDPM

Epoch 190/196



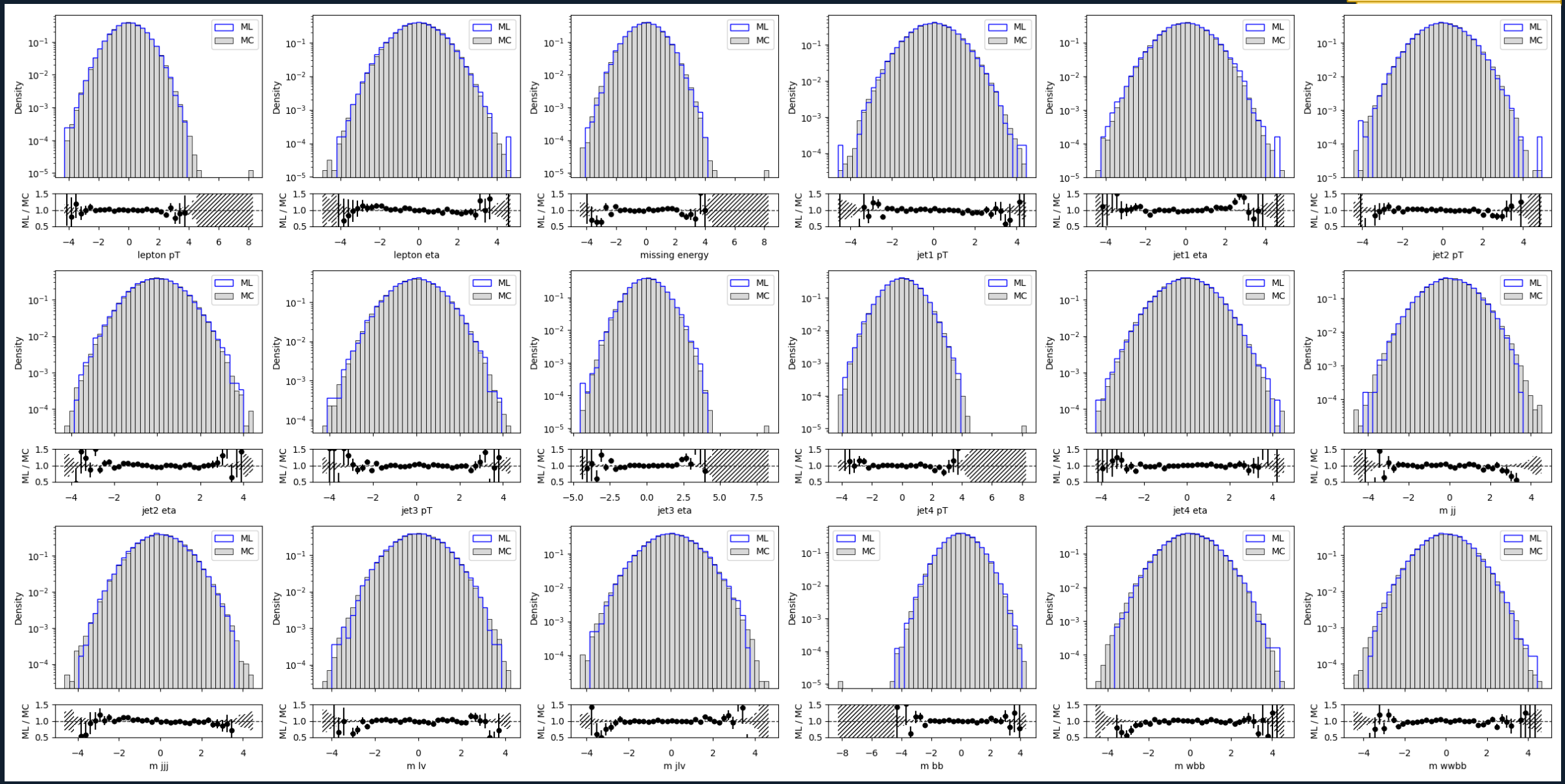
Training samples snapshots for EDM xl

Epoch 0/190



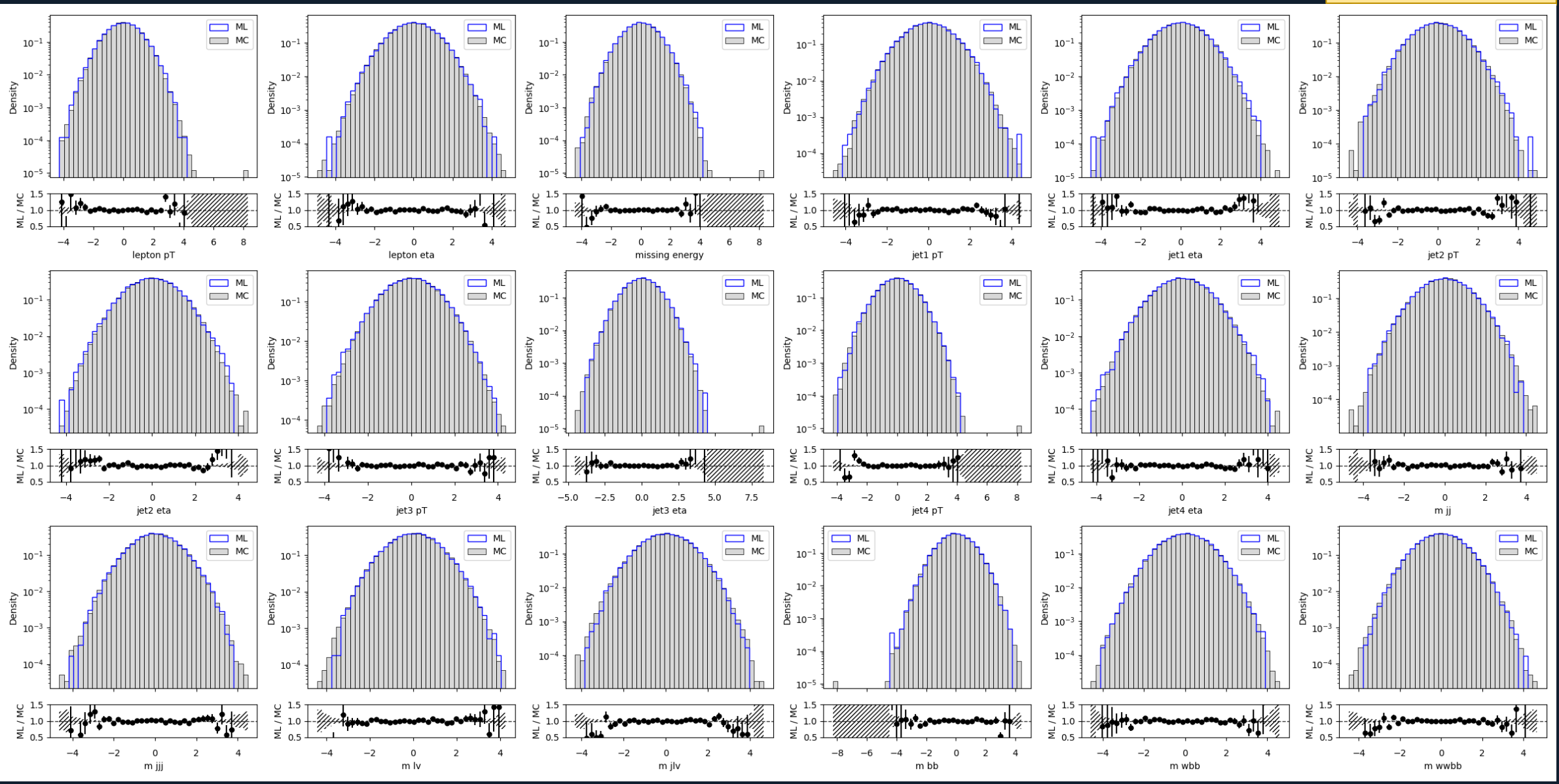
Training samples snapshots for EDM xl

Epoch 96/190



Training samples snapshots for EDM xl

Epoch 192/196

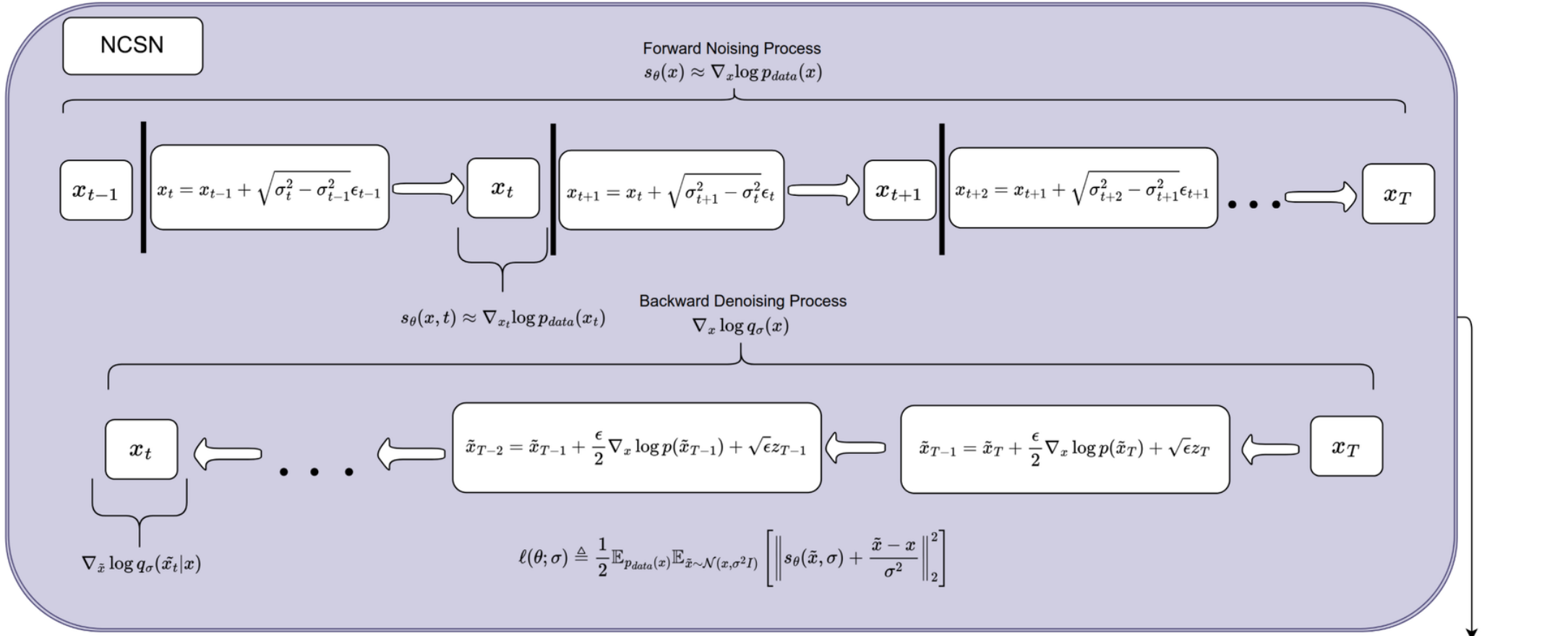


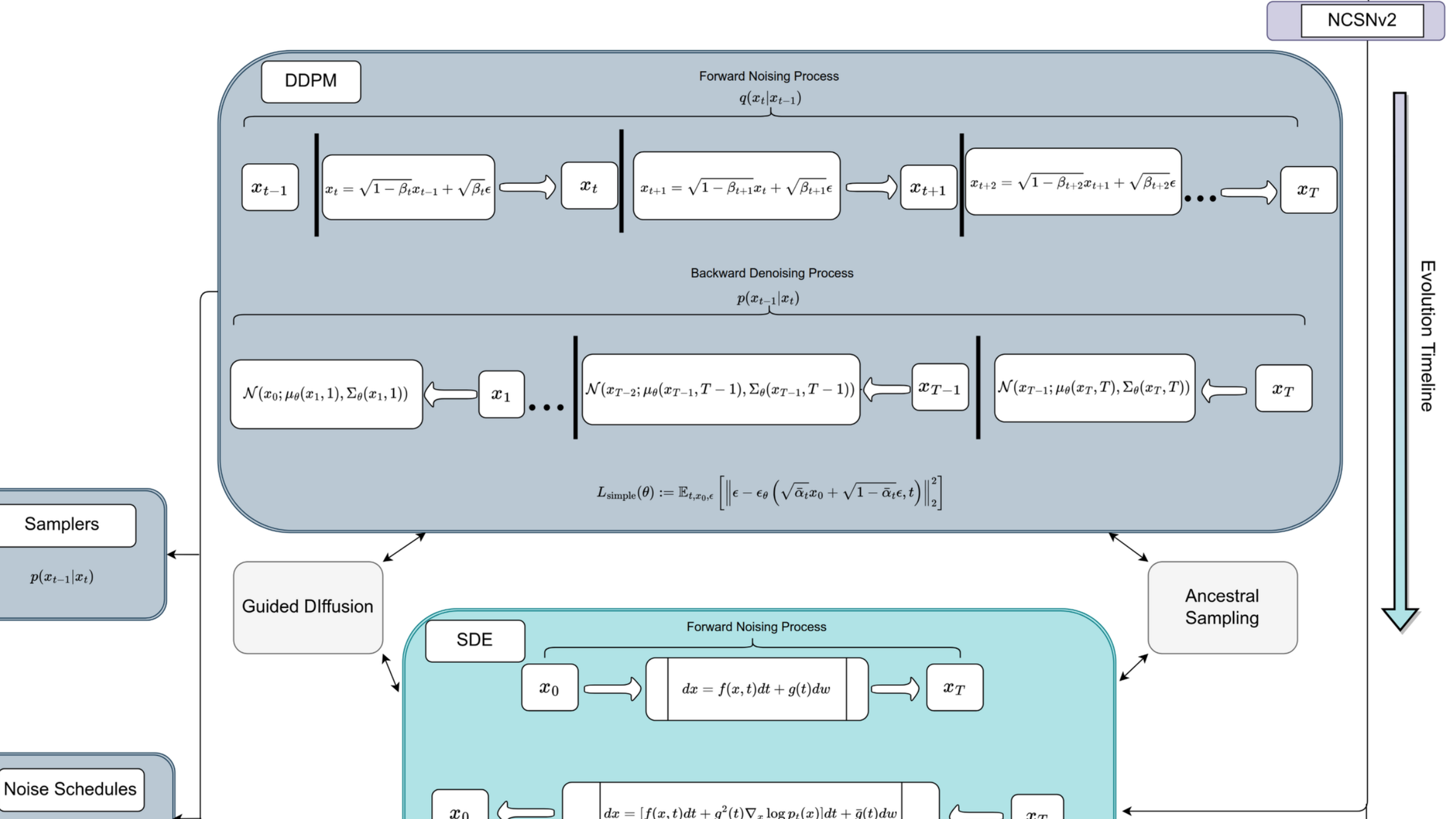
Parameters/SDE models overview

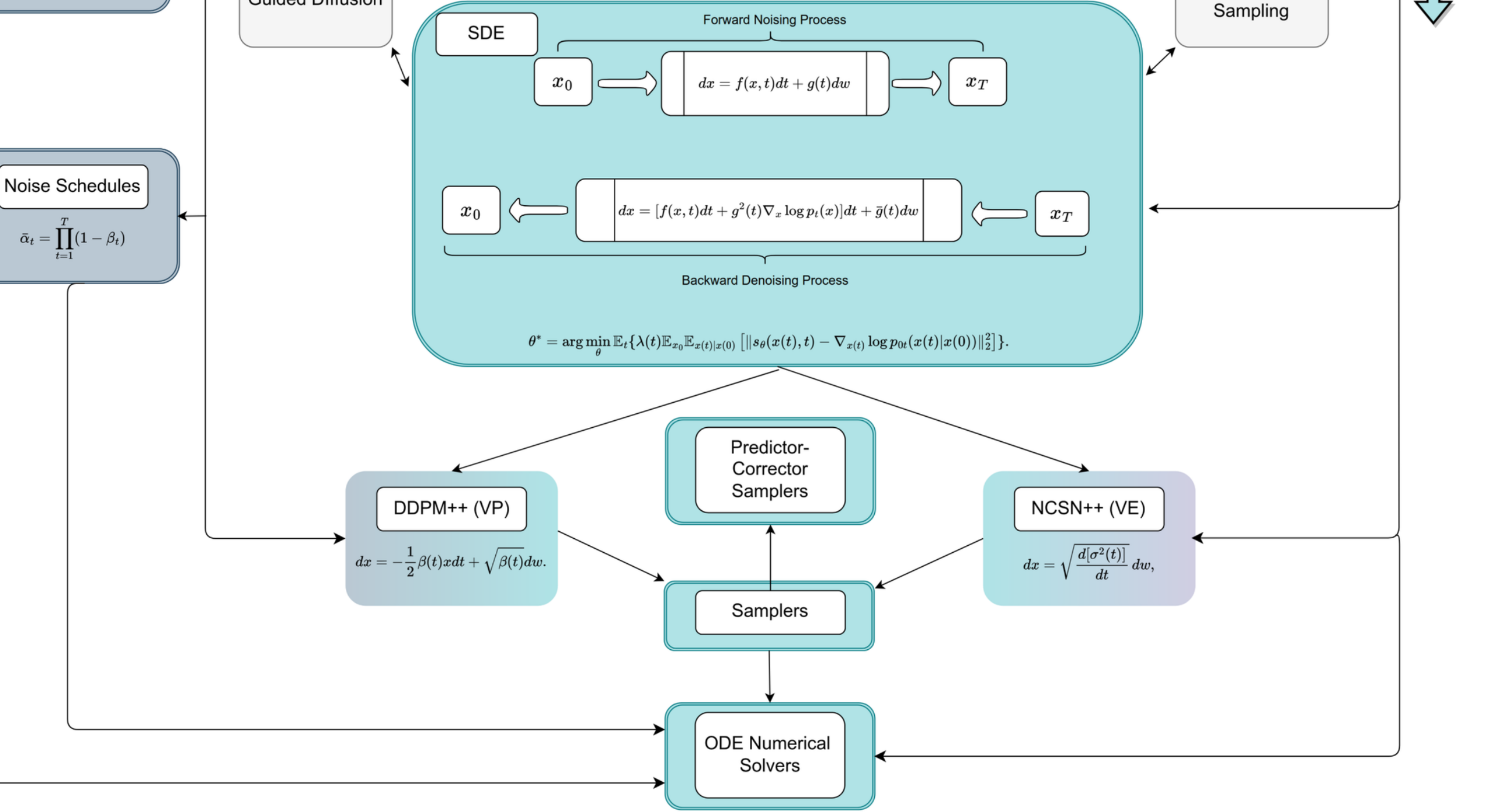
	VP [49]	VE [49]	iDDPM [37] + DDIM [47]	Ours ("EDM")
Sampling (Section 3)				
ODE solver	Euler	Euler	Euler	2nd order Heun
Time steps $t_{i<N}$	$1 + \frac{i}{N-1}(\epsilon_s - 1)$	$\sigma_{max}^2 \left(\sigma_{min}^2/\sigma_{max}^2\right)^{\frac{i}{N-1}}$	$u_{\lfloor j_0 + \frac{M-1-j_0}{N-1} i + \frac{1}{2} \rfloor}$, where $u_M = 0$, $u_{j-1} = \sqrt{\frac{u_j^2+1}{\max(\bar{\alpha}_{j-1}/\bar{\alpha}_j, C_1)} - 1}$	$\left(\sigma_{max}^{\frac{1}{\rho}} + \frac{i}{N-1}(\sigma_{min}^{\frac{1}{\rho}} - \sigma_{max}^{\frac{1}{\rho}})\right)^{\rho}$
Schedule $\sigma(t)$	$\sqrt{e^{\frac{1}{2}\beta_d t^2 + \beta_{min} t} - 1}$	$\sqrt{t}$	t	t
Scaling $s(t)$	$1/\sqrt{e^{\frac{1}{2}\beta_d t^2 + \beta_{min} t}}$	1	1	1
Network and preconditioning (Section 5)				
Architecture of F_θ	DDPM++	NCSN++	DDPM	(any)
Skip scaling $c_{skip}(\sigma)$	1	1	1	$\sigma_{data}^2/\left(\sigma^2 + \sigma_{data}^2\right)$
Output scaling $c_{out}(\sigma)$	$-\sigma$	σ	$-\sigma$	$\sigma \cdot \sigma_{data}/\sqrt{\sigma_{data}^2 + \sigma^2}$
Input scaling $c_{in}(\sigma)$	$1/\sqrt{\sigma^2+1}$	1	$1/\sqrt{\sigma^2+1}$	$1/\sqrt{\sigma^2 + \sigma_{data}^2}$
Noise cond. $c_{noise}(\sigma)$	$(M-1)\,\sigma^{-1}(\sigma)$	$\ln(\frac{1}{2}\sigma)$	$M-1-\arg\min_j \lvert u_j - \sigma \rvert$	$\frac{1}{4}\ln(\sigma)$
Training (Section 5)				
Noise distribution	$\sigma^{-1}(\sigma) \sim \mathcal{U}(\epsilon_t, 1)$	$\ln(\sigma) \sim \mathcal{U}(\ln(\sigma_{min}), \ln(\sigma_{max}))$	$\sigma = u_j,\ \ j \sim \mathcal{U}\{0, M-1\}$	$\ln(\sigma) \sim \mathcal{N}(P_{mean}, P_{std}^2)$
Loss weighting $\lambda(\sigma)$	$1/\sigma^2$	$1/\sigma^2$	$1/\sigma^2$ (note: *)	$\left(\sigma^2 + \sigma_{data}^2\right)/(\sigma \cdot \sigma_{data})^2$
Parameters	$\beta_d = 19.9, \beta_{min} = 0.1$ $\epsilon_s = 10^{-3}, \epsilon_t = 10^{-5}$ $M = 1000$	$\sigma_{min} = 0.02$ $\sigma_{max} = 100$	$\bar{\alpha}_j = \sin^2(\frac{\pi}{2}\frac{j}{M(C_2+1)})$ $C_1 = 0.001, C_2 = 0.008$ $M = 1000, j_0 = 8^\dagger$	$\sigma_{min} = 0.002, \sigma_{max} = 80$ $\sigma_{data} = 0.5, \rho = 7$ $P_{mean} = -1.2, P_{std} = 1.2$

* iDDPM also employs a second loss term L_{vlb} † In our tests, $j_0 = 8$ yielded better FID than $j_0 = 0$ used by iDDPM

Diffusion models general overview




NCSNv2
DDPM
Forward Noising Process
$q(x_t|x_{t-1})$
x_{t-1}
$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon$
x_t
$x_{t+1} = \sqrt{1-\beta_{t+1}}x_t + \sqrt{\beta_{t+1}}\epsilon$
x_{t+1}
$x_{t+2} = \sqrt{1-\beta_{t+2}}x_{t+1} + \sqrt{\beta_{t+2}}\epsilon$
x_T
Backward Denoising Process
$p(x_{t-1}|x_t)$
$\mathcal{N}(x_0; \mu_\theta(x_1, 1), \Sigma_\theta(x_1, 1))$
x_1
$\mathcal{N}(x_{T-2}; \mu_\theta(x_{T-1}, T-1), \Sigma_\theta(x_{T-1}, T-1))$
x_{T-1}
$\mathcal{N}(x_{T-1}; \mu_\theta(x_T, T), \Sigma_\theta(x_T, T))$
x_T
$L_{\text{simple}}(\theta) := \mathbb{E}_{t,x_0,\epsilon}\left[\left\|\epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t\right)\right\|_2^2\right]$
Evolution Timeline
Samplers
$p(x_{t-1}|x_t)$
Guided DIffusion
Ancestral Sampling
SDE
Forward Noising Process
x_0
$dx = f(x,t)dt + g(t)dw$
x_T
x_0
$dx = [f(x,t)dt + g^2(t)\nabla_x \log p_t(x)]dt + \bar{g}(t)dw$
x_T
Noise Schedules


Guided Diffusion
Sampling
SDE
Forward Noising Process
x_0
$dx = f(x,t)dt + g(t)dw$
x_T
Noise Schedules
$\bar{\alpha}_t = \prod_{t=1}^{T}(1-\beta_t)$
x_0
$dx = [f(x,t)dt + g^2(t)\nabla_x \log p_t(x)]dt + \bar{g}(t)dw$
x_T
Backward Denoising Process
$\theta^* = \arg\min_\theta \mathbb{E}_t\{\lambda(t)\mathbb{E}_{x_0}\mathbb{E}_{x(t)|x(0)}\left[\|s_\theta(x(t),t) - \nabla_{x(t)}\log p_{0t}(x(t)|x(0))\|_2^2\right]\}.$
Predictor-Corrector Samplers
DDPM++ (VP)
$dx = -\frac{1}{2}\beta(t)xdt + \sqrt{\beta(t)}dw.$
NCSN++ (VE)
$dx = \sqrt{\frac{d[\sigma^2(t)]}{dt}}\,dw,$
Samplers
ODE Numerical Solvers